

APUNTES

Programación de Servicios y Procesos

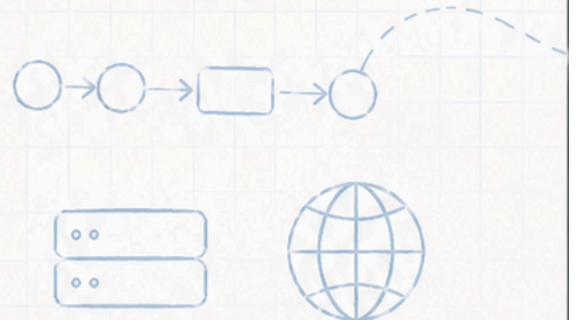
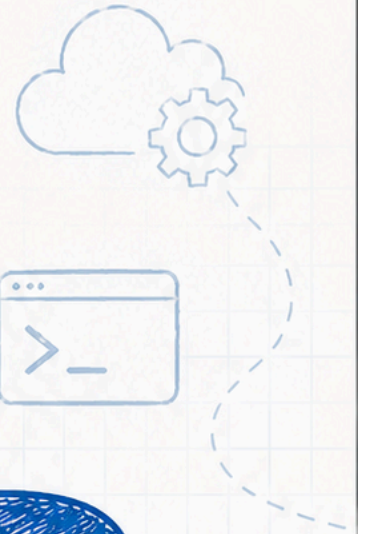
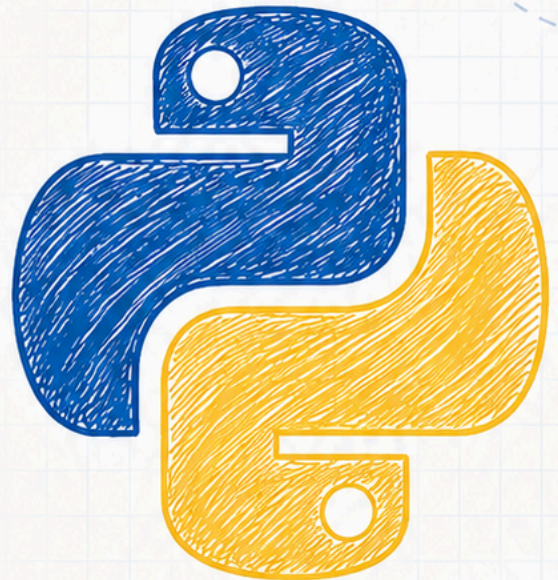
(Con Python)

```
1  import time
2  import socket
3  from multiprocessing import Process
4
5  def trabajo(nombre):
6      print(f"Iniciando {nombre}...")
7      time.sleep(2)
8      print(f"{nombre} finalizado.")
9
10 if __name__ == "__main__":
11     p = Process(target=trabajo, args=("Proceso",))
12     p.start()
12     p.join()
```

importaciones

definición de función

ejecución del proceso



Por
Sergi García Barea

licencia
CC BY SA



Contents

- [Inicio](#)
 -  [Unidades](#)
 -  [Ejercicios](#)
 -  [Licencia](#)
- [TEMA 01 — Procesos y Subprocess](#)
 - [TEMA 01 — Procesos y Subprocess \(RA1\)](#)
 - [Índice](#)
 - [¿Qué es un proceso?](#)
 - [Estados de un proceso](#)
 - [Paralela vs Distribuida](#)
 - [subprocess.run\(\) — lanzar y esperar](#)
 - [subprocess.Popen\(\) — lanzar y seguir](#)
 - [Comunicación con procesos](#)
 - [Be the code, my friend, my friend — Abrir bloc de notas y calculadora](#)
 -  [El ring de los conceptos — run\(\) vs Popen\(\)](#)
 - [Preguntas tontas — Procesos](#)
 -  [Aprieta el lápiz](#)
 - [RAs cubiertos y criterios de evaluación](#)
- [TEMA 02 — Hilos Fundamentos](#)
 - [TEMA 02 — Hilos Fundamentos \(RA2\)](#)
 - [Índice](#)
 - [¿Qué es un hilo?](#)
 - [Hilos vs Procesos](#)
 - [Crear y lanzar hilos](#)
 - [join\(\) — esperar a que termine](#)
 - [Hilos daemon](#)
 - [Timer — ejecutar algo después de un tiempo](#)
 - [El GIL y sus consecuencias](#)
 - [Estados de un hilo](#)
 - [Be the code, my friend, my friend — El hilo viajero](#)
 - [Be the code, my friend, my friend — daemon en acción](#)

-  [El ring de los conceptos — Hilo normal vs Hilo daemon](#)
- [Preguntas tontas — Hilos](#)
-  [Aprieta el lápiz](#)
- [RAs cubiertos y criterios de evaluación](#)
- [TEMA 03 — Sincronización entre Hilos](#)
 - [TEMA 03 — Sincronización entre Hilos \(RA2\)](#)
 - [Índice](#)
 - [El problema de la condición de carrera](#)
 - [Lock — exclusión mutua](#)
 - [Be the code, my friend, my friend — Contador con y sin Lock](#)
 - [RLock — Lock reentrante](#)
 - [Semaphore — acceso limitado](#)
 - [Barrier — sincronización de grupo](#)
 - [Condition — productor-consumidor](#)
 -  [El ring de los conceptos — Lock vs Semáforo vs Barrera](#)
 - [Be the code, my friend, my friend — Pool Puzzle de sincronización](#)
 - [Preguntas tontas — Sincronización](#)
 -  [Aprieta el lápiz](#)
 - [RAs cubiertos y criterios de evaluación](#)
- [TEMA 04 — Sockets TCP](#)
 - [TEMA 04 — Sockets TCP \(RA3\)](#)
 - [Índice](#)
 - [¿Qué es un socket?](#)
 - [TCP — orientado a conexión](#)
 - [Servidor TCP — ciclo de vida](#)
 - [Cliente TCP — ciclo de vida](#)
 - [Be the code, my friend, my friend — Mano a mano TCP](#)
 - [SO_REUSEADDR — no más “Address already in use”](#)
 - [Errores comunes y cómo manejarlos](#)
 - [Non-blocking y timeouts](#)
 -  [Pool Puzzle — El ciclo de vida del socket](#)
 -  [El ring de los conceptos — Servidor vs Cliente](#)
 - [Preguntas tontas — Sockets TCP](#)

-  [Aprieta el lápiz](#)
- [RAs cubiertos y criterios de evaluación](#)
- [TEMA 05 — Sockets UDP y Protocolos](#)
 - [TEMA 05 — Sockets UDP y Protocolos \(RA3\)](#)
 - [Índice](#)
 - [UDP — sin conexión](#)
 - [Servidor UDP](#)
 - [Cliente UDP](#)
 -  [El ring de los conceptos — TCP vs UDP](#)
 - [HTTP desde cero — el protocolo que mueve la web](#)
 - [Be the code, my friend, my friend — Cliente HTTP manual](#)
 - [NTP — ¿qué hora es en Internet?](#)
 - [Preguntas tontas — UDP y Protocolos](#)
 -  [Aprieta el lápiz](#)
 - [RAs cubiertos y criterios de evaluación](#)
- [TEMA 06 — APIs REST y HTTP](#)
 - [TEMA 06 — APIs REST y HTTP \(RA4a-b\)](#)
 - [Índice](#)
 - [¿Qué es una API?](#)
 - [REST — el estándar de las APIs modernas](#)
 - [HTTP — los verbos](#)
 - [Códigos de estado — ¿salió bien o mal?](#)
 - [La librería requests](#)
 - [JSON — el idioma de las APIs](#)
 - [Cabeceras y parámetros](#)
 - [Be the code, my friend, my friend — Petición paso a paso](#)
 -  [Pool Puzzle — Petición API con errores](#)
 -  [El ring de los conceptos — GET vs POST vs PUT vs DELETE](#)
 - [Preguntas tontas — APIs y REST](#)
 -  [Aprieta el lápiz](#)
 - [RAs cubiertos y criterios de evaluación](#)
- [TEMA 07 — APIs Comerciales](#)
 - [TEMA 07 — APIs Comerciales \(RA4a-b\)](#)

- [Índice](#)
- [API Key — el carnet de identidad](#)
- [Variables de entorno con dotenv](#)
- [OpenWeatherMap — el tiempo en tu ciudad](#)
- [Be the code, my friend, my friend — Consulta meteorológica](#)
- [OpenAI — el cerebro artificial](#)
- [Errores y rate limiting](#)
- [Reintentos inteligentes \(backoff\)](#)
-  [El ring de los conceptos — REST vs SOAP vs GraphQL](#)
- [Herramientas para probar APIs](#)
- [Preguntas tontas — APIs Comerciales](#)
-  [Aprieta el lápiz](#)
- [RAs cubiertos y criterios de evaluación](#)
- [TEMA 08 — Hash y Cifrado Clásico](#)
 - [TEMA 08 — Hash y Cifrado Clásico \(RA5\)](#)
 - [Índice](#)
 - [Principios de seguridad](#)
 - [Hash — la huella digital](#)
 - [MD5, SHA-1, SHA-256 — ¿cuál usar?](#)
 - [Be the code, my friend, my friend — Hash de una contraseña](#)
 - [Hash con sal](#)
 -  [Pool Puzzle — Verificación de contraseña con hash](#)
 -  [El ring de los conceptos — Hash vs Cifrado](#)
 - [Preguntas tontas — Hash](#)
 - [Cifrado César — el abuelo de la criptografía](#)
 - [Be the code, my friend, my friend — César paso a paso](#)
 -  [Aprieta el lápiz](#)
 - [RAs cubiertos y criterios de evaluación](#)
- [TEMA 09 — Cifrado Moderno](#)
 - [TEMA 09 — Cifrado Moderno \(RA5\)](#)
 - [Índice](#)
 - [Cifrado simétrico — AES](#)
 - [Componentes de AES](#)

- [Be the code, my friend, my friend — AES paso a paso](#)
- [Cifrado asimétrico — RSA](#)
-  [El ring de los conceptos — AES vs RSA](#)
- [Cifrado híbrido — lo mejor de ambos mundos](#)
- [Firmas digitales — demostrar autoría](#)
- [Roles y RBAC](#)
- [Be the code, my friend, my friend — Firma y verificación](#)
- [Preguntas tontas — Cifrado Moderno](#)
-  [Aprieta el lápiz](#)
- [RAs cubiertos y criterios de evaluación](#)
- [TEMA 10 — Servidores Concurrentes](#)
 - [TEMA 10 — Servidores Concurrentes \(RA4c-d\)](#)
 - [Índice](#)
 - [El problema del servidor secuencial](#)
 - [Be the code, my friend, my friend — Servidor secuencial vs concurrente](#)
 - [Servidor con hilos \(threading\)](#)
 - [ThreadPoolExecutor — control de recursos](#)
 -  [Benchmark — Secuencial vs Hilos vs Pool](#)
 -  [Pool Puzzle — Servidor con ThreadPool](#)
 -  [El ring de los conceptos — Hilo por cliente vs ThreadPool](#)
 - [Preguntas tontas — Concurrencia](#)
 -  [Aprieta el lápiz](#)
 - [RAs cubiertos y criterios de evaluación](#)
- [TEMA 11 — Asyncio y Disponibilidad](#)
 - [TEMA 11 — Asyncio y Disponibilidad \(RA4e-g\)](#)
 - [Índice](#)
 - [El problema de esperar](#)
 - [Asyncio — fundamentos](#)
 - [Corrutinas y event loop](#)
 - [Servidor con asyncio](#)
 - [Be the code, my friend, my friend — Asyncio paso a paso](#)
 - [Disponibilidad — heartbeat](#)
 - [Reintentos con backoff](#)

- [Timeout en servidor](#)
- [🔴 El ring de los conceptos — Threads vs Asyncio](#)
- [Preguntas tontas — Asyncio](#)
- [✎ Aprieta el lápiz](#)
- [RAs cubiertos y criterios de evaluación](#)
- [✅ INICIAL RESUELTO 1 — Procesos y Subprocess](#)
- [🟢 INICIAL POR RESOLVER 1 — Procesos y Subprocess](#)
- [👉 INTERMEDIO RESUELTO 1 — Procesos y Subprocess](#)
- [📄 INTERMEDIO POR RESOLVER 1 — Procesos y Subprocess](#)
- [★ AVANZADO 1 — Procesos y Subprocess](#)
 - [★ AVANZADO 01 — Procesos y Subprocess](#)
- [✅ INICIAL RESUELTO 2 — Hilos Fundamentos](#)
- [🟢 INICIAL POR RESOLVER 2 — Hilos Fundamentos](#)
- [👉 INTERMEDIO RESUELTO 2 — Hilos Fundamentos](#)
- [📄 INTERMEDIO POR RESOLVER 2 — Hilos Fundamentos](#)
- [★ AVANZADO 2 — Hilos Fundamentos](#)
 - [★ AVANZADO 02 — Hilos Fundamentos](#)
- [✅ INICIAL RESUELTO 3 — Sincronización entre Hilos](#)
- [🟢 INICIAL POR RESOLVER 3 — Sincronización entre Hilos](#)
- [👉 INTERMEDIO RESUELTO 3 — Sincronización entre Hilos](#)
- [📄 INTERMEDIO POR RESOLVER 3 — Sincronización entre Hilos](#)
- [★ AVANZADO 3 — Sincronización entre Hilos](#)
 - [★ AVANZADO 03 — Sincronización entre Hilos](#)
- [✅ INICIAL RESUELTO 4 — Sockets TCP](#)
- [🟢 INICIAL POR RESOLVER 4 — Sockets TCP](#)
- [👉 INTERMEDIO RESUELTO 4 — Sockets TCP](#)
- [📄 INTERMEDIO POR RESOLVER 4 — Sockets TCP](#)
- [★ AVANZADO 4 — Sockets TCP](#)
 - [★ AVANZADO 04 — Sockets TCP](#)
- [✅ INICIAL RESUELTO 5 — Sockets UDP y Protocolos](#)
- [🟢 INICIAL POR RESOLVER 5 — Sockets UDP y Protocolos](#)
- [👉 INTERMEDIO RESUELTO 5 — Sockets UDP y Protocolos](#)
- [📄 INTERMEDIO POR RESOLVER 5 — Sockets UDP y Protocolos](#)

- [!\[\]\(694fcb4611893e9db5249daba48abfc1_img.jpg\) AVANZADO 5 — Sockets UDP y Protocolos](#)
 - [!\[\]\(8ec8d5dc48934930a762fecf6ecbe179_img.jpg\) AVANZADO 05 — Sockets UDP y Protocolos](#)
- [!\[\]\(c34a15e67573dae8fbb88f4cbfb0f2e9_img.jpg\) INICIAL RESUELTO 6 — APIs REST y HTTP](#)
- [!\[\]\(41f06fdeabb4e5a71d06fe8f32a46127_img.jpg\) INICIAL POR RESOLVER 6 — APIs REST y HTTP](#)
- [!\[\]\(18eb66208e65404cce5042d73cf0a851_img.jpg\) INTERMEDIO RESUELTO 6 — APIs REST y HTTP](#)
- [!\[\]\(14a9d4de9e6699d41b68e8807e2d5f76_img.jpg\) INTERMEDIO POR RESOLVER 6 — APIs REST y HTTP](#)
- [!\[\]\(415790129e00c225ba52b81c8addfb14_img.jpg\) AVANZADO 6 — APIs REST y HTTP](#)
 - [!\[\]\(fa8e43d6f5da9cf596808674ced6c198_img.jpg\) AVANZADO 06 — APIs REST y HTTP](#)
- [!\[\]\(2b564e327fe9708ac2f9320a9ae84c76_img.jpg\) INICIAL RESUELTO 7 — APIs Comerciales](#)
- [!\[\]\(484cd33a03c33977d2fcf6bb9cc02435_img.jpg\) INICIAL POR RESOLVER 7 — APIs Comerciales](#)
- [!\[\]\(c885083f23ac65632c3cf77b16f7a193_img.jpg\) INTERMEDIO RESUELTO 7 — APIs Comerciales](#)
- [!\[\]\(20f0f5805f0a60636883bdd13c58dc31_img.jpg\) INTERMEDIO POR RESOLVER 7 — APIs Comerciales](#)
- [!\[\]\(90f7aa1b3da6a942e186a7e3fdaaf44b_img.jpg\) AVANZADO 7 — APIs Comerciales](#)
 - [!\[\]\(b2b96e5d9b571004907039c0a8a75b54_img.jpg\) AVANZADO 07 — APIs Comerciales](#)
- [!\[\]\(6285cf242551105b2a630c622e0c55ae_img.jpg\) INICIAL RESUELTO 8 — Hash y Cifrado Clásico](#)
- [!\[\]\(ea61eebb01e76a95ff1c56cf17f53608_img.jpg\) INICIAL POR RESOLVER 8 — Hash y Cifrado Clásico](#)
- [!\[\]\(bef5c8d7250866df258df26efebb2d6b_img.jpg\) INTERMEDIO RESUELTO 8 — Hash y Cifrado Clásico](#)
- [!\[\]\(1855a3a8d9a41a3f4f1dc8d294fd804e_img.jpg\) INTERMEDIO POR RESOLVER 8 — Hash y Cifrado Clásico](#)
- [!\[\]\(4bff6b8888da539b37a899aef567f1be_img.jpg\) AVANZADO 8 — Hash y Cifrado Clásico](#)
 - [!\[\]\(1a1b0261b18044c47100576c25c435f9_img.jpg\) AVANZADO 08 — Hash y Cifrado Clásico](#)
- [!\[\]\(7b1db38756a02e50672c39a699367588_img.jpg\) INICIAL RESUELTO 9 — Cifrado Moderno](#)
- [!\[\]\(5092e0d5a46a35d887ea84e10efda211_img.jpg\) INICIAL POR RESOLVER 9 — Cifrado Moderno](#)
- [!\[\]\(222624bfbcd41cb4b8e992f2fb4d1d5d_img.jpg\) INTERMEDIO RESUELTO 9 — Cifrado Moderno](#)
- [!\[\]\(90ca981cfd6ce35151faff84419b8399_img.jpg\) INTERMEDIO POR RESOLVER 9 — Cifrado Moderno](#)
- [!\[\]\(4cdd93c76f3bb2c7ca3858ab7830b87e_img.jpg\) AVANZADO 9 — Cifrado Moderno](#)
 - [!\[\]\(cad3a662fcdef3adde1636c9b3b708d8_img.jpg\) AVANZADO 09 — Cifrado Moderno](#)
- [!\[\]\(45e5f22e5e91d2d2349088139ffd9439_img.jpg\) INICIAL RESUELTO 10 — Servidores Concurrentes](#)
- [!\[\]\(23345d74f4e8714038e154b94c68c7b8_img.jpg\) INICIAL POR RESOLVER 10 — Servidores Concurrentes](#)
- [!\[\]\(e04c62507b16f6eaa81ce61047658f95_img.jpg\) INTERMEDIO RESUELTO 10 — Servidores Concurrentes](#)
- [!\[\]\(5d263efc5b89ddeb3f4db4d8fa8ce844_img.jpg\) INTERMEDIO POR RESOLVER 10 — Servidores Concurrentes](#)
- [!\[\]\(cc1dba3bdf2c04aaa2540e768741044f_img.jpg\) AVANZADO 10 — Servidores Concurrentes](#)
 - [!\[\]\(ca155c01665f65d374236c31be673308_img.jpg\) AVANZADO 10 — Servidores Concurrentes](#)
- [!\[\]\(82249cbd3998e114ccbd040d722049d8_img.jpg\) INICIAL RESUELTO 11 — Asyncio y Disponibilidad](#)

-  [INICIAL POR RESOLVER 11 — Asyncio y Disponibilidad](#)
-  [INTERMEDIO RESUELTO 11 — Asyncio y Disponibilidad](#)
-  [INTERMEDIO POR RESOLVER 11 — Asyncio y Disponibilidad](#)
-  [AVANZADO 11 — Asyncio y Disponibilidad](#)
 -  [AVANZADO 11 — Asyncio y Disponibilidad](#)

Inicio

APUNTES

Programación de Servicios y Procesos

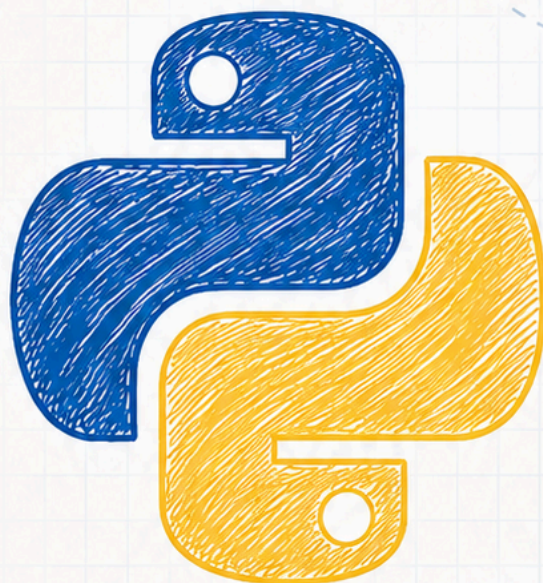
(Con Python)

```
1 import time
2 import socket
3 from multiprocessing import Process
4
5 def trabajo(nombre):
6     print(f"Iniciando {nombre}...")
7     time.sleep(2)
8     print(f"{nombre} finalizado.")
9
10 if __name__ == "__main__":
11     p = Process(target=trabajo, args=("Proceso",))
12     p.start()
13     p.join()
```

importaciones

definición de función

ejecución del proceso



Por
Sergi García Barea

licencia
CC BY SA



[🚀 Empezar por la Unidad 1](#)[🐙 Ver en GitHub](#)

Sergi Garcia Barea — CC BY-SA 4.0

[↓ ¿Prefieres estudiar sin conexión? Descarga el curso completo](#)[PDF](#)[EPUB](#)

Unidades

UNIDAD 1

RA1

Procesos y Subprocess

Procesos, `subprocess.run`, `subprocess.Popen`, comunicación entre procesos, paralela vs distribuida.

[↩ Ver unidad ➡](#)

UNIDAD 2

RA2

Hilos Fundamentos

Crear hilos con `threading.Thread`, `join`, `daemon`, `Timer`, GIL, estados del hilo.

[↩ Ver unidad ➡](#)

UNIDAD 3

RA2

Sincronización entre Hilos

`Lock`, `Semaphore`, `Barrier`, `Condition`, `RLock`, productor-consumidor.

[↩ Ver unidad ➡](#)

UNIDAD 4

RA3

Sockets TCP

TCP cliente/servidor, `socket`, `bind`, `listen`, `accept`, errores, `SO_REUSEADDR`.

[↩ Ver unidad ➡](#)

UNIDAD 5

RA3

Sockets UDP y Protocolos

UDP, HTTP desde cero, NTP, TCP vs UDP, protocolos de aplicación.

[↩ Ver unidad ➡](#)

UNIDAD 6

RA4a-b

APIs REST y HTTP

REST, métodos HTTP, `requests`, JSON, códigos de estado, consumo de APIs.

[↩ Ver unidad ➡](#)

UNIDAD 7

RA4a-b

UNIDAD 8

RA5

APIs Comerciales

OpenWeatherMap, OpenAI, `python-dotenv`, `rate limit`, manejo de errores, autenticación.

👉 Ver unidad 👈

Hash y Cifrado Clásico

Hash, MD5, SHA, Cifrado César, principios de seguridad, esteganografía básica.

👉 Ver unidad 👈

UNIDAD 9

RA5

Cifrado Moderno

AES, RSA, cifrado híbrido, firmas digitales, RBAC, `pycryptodome`.

👉 Ver unidad 👈

UNIDAD 10

RA4c-d

Servidores Concurrentes

ThreadPool, servidor multihilo, `concurrent.futures`, balanceo de carga básico.

👉 Ver unidad 👈

UNIDAD 11

RA4e-g

Asyncio y Disponibilidad

`asyncio`, `async / await`, heartbeat, reintentos con backoff, timeouts, alta disponibilidad.

👉 Ver unidad 👈



Ejercicios

UNIDAD 1

✅ Inicial resuelto

● Inicial por resolver

👉 Intermedio resuelto

📄 Intermedio por resolver

★ Avanzado

UNIDAD 2

✅ Inicial resuelto

● Inicial por resolver

👉 Intermedio resuelto

📄 Intermedio por resolver

★ Avanzado

UNIDAD 3

✅ Inicial resuelto

● Inicial por resolver

👉 Intermedio resuelto

📄 Intermedio por resolver

★ Avanzado

UNIDAD 4

✅ Inicial resuelto

● Inicial por resolver

👉 Intermedio resuelto

📄 Intermedio por resolver

★ Avanzado

UNIDAD 5

- ✓ Inicial resuelto
- Inicial por resolver
- 👉 Intermedio resuelto
- 📄 Intermedio por resolver
- ★ Avanzado

UNIDAD 6

- ✓ Inicial resuelto
- Inicial por resolver
- 👉 Intermedio resuelto
- 📄 Intermedio por resolver
- ★ Avanzado

UNIDAD 7

- ✓ Inicial resuelto
- Inicial por resolver
- 👉 Intermedio resuelto
- 📄 Intermedio por resolver
- ★ Avanzado

UNIDAD 8

- ✓ Inicial resuelto
- Inicial por resolver
- 👉 Intermedio resuelto
- 📄 Intermedio por resolver
- ★ Avanzado

UNIDAD 9

- ✓ Inicial resuelto
- Inicial por resolver
- 👉 Intermedio resuelto
- 📄 Intermedio por resolver
- ★ Avanzado

UNIDAD 10

- ✓ Inicial resuelto
- Inicial por resolver
- 👉 Intermedio resuelto
- 📄 Intermedio por resolver
- ★ Avanzado

UNIDAD 11

- ✓ Inicial resuelto
- Inicial por resolver
- 👉 Intermedio resuelto
- 📄 Intermedio por resolver
- ★ Avanzado



Licencia



CC BY-SA 4.0 — *Sergi Garcia Barea*

[Licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional](https://creativecommons.org/licenses/by-sa/4.0/)

Puedes compartir y adaptar el material para cualquier propósito, incluso comercial, siempre que reconozcas la autoría y compartas las modificaciones bajo la misma licencia.

TEMA 01 — Procesos y Subprocess

TEMA 01 — Procesos y Subprocess (RA1)

“Un programa en el disco duro es un muerto viviente. Un proceso es ese mismo programa **vivo**, ejecutándose, ocupando memoria, consumiendo CPU.”

Compatibilidad Windows / Linux

Los ejemplos están escritos para **Windows** (notepad.exe, calc.exe, ping -n, etc.), pero los conceptos (subprocess.run, Popen, PID, communicate) son **idénticos en todos los sistemas**.

Windows	Linux / macOS
notepad.exe	gedit, nano, xed
calc.exe	gnome-calculator, bc
mspaint.exe	pinta, kolourpaint
ping -n 5 8.8.8.8	ping -c 5 8.8.8.8
ipconfig	ip addr, ifconfig
where python	which python
cmd /c mkdir	mkdir (es ejecutable real)
start http://...	xdg-open http://...
dir	ls

Sustituye el comando correspondiente y el código funciona igual.

Índice

1. [¿Qué es un proceso?](#)
2. [Estados de un proceso](#)
3. [Paralela vs Distribuida](#)
4. [subprocess.run\(\) — lanzar y esperar](#)

5. [subprocess.Popen\(\) — lanzar y seguir](#)
6. [Comunicación con procesos](#)
7. [Be the code, my friend, my friend — Abrir bloc de notas y calculadora](#)
8.  [El ring de los conceptos — run\(\) vs Popen\(\)](#)
9. [Preguntas tontas — Procesos](#)
10.  [Aprieta el lápiz](#)
11. [RAs cubiertos y criterios de evaluación](#)

¿Qué es un proceso?

Un **proceso** es un programa en ejecución con su propio espacio de memoria, recursos (archivos abiertos, sockets) y un identificador único llamado **PID**.

```
import os
print(f"Este proceso se llama PID {os.getpid()}")
```

Características

Propiedad	Descripción
PID	Identificador único numérico
Memoria propia	Cada proceso tiene su espacio de direcciones aislado
Recursos	Archivos, sockets, manejadores
Contexto	Estado de la CPU, registros, contador de programa
Comunicación	Necesita mecanismos externos (pipes, sockets, archivos)

“Si un proceso se cuelga, los demás no se enteran. Cada uno vive en su burbuja de memoria.”

Estados de un proceso



Estado	Qué significa
NUEVO	El proceso acaba de ser creado
LISTO	Preparado para ejecutar, esperando que la CPU esté libre
EJECUCIÓN	La CPU está ejecutando sus instrucciones
BLOQUEADO	Esperando un recurso (I/O, socket, sleep)
TERMINADO	El proceso ha finalizado

Paralela vs Distribuida

Concepto	Qué significa
Paralela	Varias tareas ejecutándose a la vez en múltiples CPUs/núcleos
Distribuida	Varias tareas ejecutándose en múltiples máquinas conectadas por red
Concurrencia	Varias tareas avanzando (no necesariamente a la vez, pueden turnarse)

Ejemplos cotidianos

- **Paralela:** 4 hilos de cocina, cada uno friendo un huevo en su sartén (4 CPUs)
- **Distribuida:** 4 restaurantes en 4 ciudades, todos cocinando el mismo menú

- **Concurrente:** 1 cocinero que va friendo huevos de 3 pedidos, alternando

```
# Paralela con multiprocessing (varios CPUs)
from multiprocessing import Pool
def cuadrado(n):
    return n * n

with Pool(4) as p: # 4 procesos en paralelo
    print(p.map(cuadrado, [1, 2, 3, 4]))
```

subprocess.run() — lanzar y esperar

La función más sencilla. Lanza un programa y **espera** a que termine.

```
import subprocess

# Comando simple
resultado = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)
print(f"Salida: {resultado.stdout}")
print(f"Código de retorno: {resultado.returncode}")
```

Parámetros importantes

Parámetro	Qué hace
capture_output=True	Captura stdout y stderr
text=True	Devuelve strings en vez de bytes
timeout=N	Lanza excepción si tarda más de N segundos
check=True	Lanza excepción si el código de retorno no es 0

```
# Con timeout y control de errores
try:
    resultado = subprocess.run(
        ["ping", "google.com", "-n", "3"],
        capture_output=True,
        text=True,
        timeout=10
    )
    print(resultado.stdout)
except subprocess.TimeoutExpired:
    print("❌ El ping tardó demasiado")
except subprocess.CalledProcessError:
    print("❌ Error en el comando")
```

subprocess.Popen() — lanzar y seguir

Popen lanza el proceso en **segundo plano** y devuelve el control inmediatamente.

```
import subprocess

# Lanzar el bloc de notas (no espera)
proceso = subprocess.Popen(["notepad.exe"])
print(f"Bloc de notas lanzado con PID {proceso.pid}")

# Podemos hacer otras cosas mientras el bloc de notas está abierto
print("Haciendo otras cosas...")

# Cuando queramos, esperamos a que termine
proceso.wait()
print("El bloc de notas se cerró")
```

Método de Popen	Qué hace
proceso.wait()	Espera a que termine (bloqueante)
proceso.poll()	Pregunta si ha terminado (no bloqueante)
proceso.terminate()	Envía señal de terminación

Método de Popen	Qué hace
<code>proceso.kill()</code>	Mata el proceso forzosamente
<code>proceso.pid</code>	PID del proceso hijo

Comunicación con procesos

```
import subprocess

# Escribir en stdin y leer stdout
proceso = subprocess.Popen(
    ["python", "-c", "print(input().upper())"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True
)

salida, _ = proceso.communicate(input="hola mundo")
print(f"El proceso respondió: {salida.strip()}")
# → "HOLA MUNDO"
```

Be the code, my friend, my friend — Abrir bloc de notas y calculadora

“Sé el programa que abre dos aplicaciones a la vez. Traza cada paso.”

```

import subprocess, time

print("🚀 Abriendo bloc de notas...")
notepad = subprocess.Popen(["notepad.exe"])
print(f"    → PID: {notepad.pid}")

print("🚀 Abriendo calculadora...")
calc = subprocess.Popen(["calc.exe"])
print(f"    → PID: {calc.pid}")

print("\nAmbos programas están abiertos.")
print("El usuario puede escribir en el bloc de notas y usar la calculadora.")

# Mientras tanto, Python no está bloqueado
for i in range(5):
    print(f"    Python haciendo cosas... ({i+1}/5)")
    time.sleep(1)

print("\nCerrando programas...")
notepad.terminate()
calc.kill()
print("Hecho 🏁")

```

Traza paso a paso:


```
1. subprocess.Popen(["notepad.exe"])
   → El SO crea un nuevo proceso
   → Windows busca notepad.exe en el PATH
   → Lo carga en memoria
   → Asigna PID 12345
   → Python recibe el control inmediatamente

2. print("PID: 12345")

3. subprocess.Popen(["calc.exe"])
   → Mismo proceso, PID 12346

4. Python hace 5 cosas mientras ambos están abiertos
   → 3 procesos independientes: Python, notepad, calc

5. notepad.terminate()
   → Envía señal de cierre al bloc de notas

6. calc.kill()
   → Mata la calculadora forzosamente
```



El ring de los conceptos — run() vs Popen()

run(): — ¡Yo soy el rey de la simplicidad! Lanzas un comando, esperas, y ¡zas! tienes el resultado.

Popen(): — Sí, pero mientras tú esperas como un muñeco, yo puedo lanzar un proceso y seguir haciendo otras cosas. ¿Para qué esperar si no hace falta?

run(): — ¿Y si necesitas el resultado? Conmigo es directo: `result.stdout` y ya. Tú necesitas `communicate()`, más vueltas...

Popen(): — Pero imagina que quieres lanzar 3 programas a la vez. Conmigo puedes lanzarlos, irte a hacer café, y luego matarlos a todos. Tú tendrías que esperar a que termine cada uno antes de lanzar el siguiente.

run(): — Vale, vale... para procesos que viven en segundo plano, eres mejor. Pero para comandos rápidos y resultados inmediatos, soy más limpio.

Moraleja: `run()` para comandos rápidos que necesitan respuesta. `Popen()` para procesos que deben vivir en segundo plano.

Preguntas tontas — Procesos

? **¿Cuántos procesos puede tener mi sistema?** Depende de la memoria RAM. Cada proceso ocupa memoria. En Windows puedes verlos en el Administrador de tareas.

? **¿Qué pasa si un proceso hijo muere?** El proceso padre puede enterarse con `proceso.wait()` o `proceso.poll()`. Si no, el hijo se convierte en **zombie** (ocupa una entrada en la tabla de procesos).

? **¿Y si el padre muere antes que el hijo?** Los hijos se convierten en **huérfanos**. En Windows, el sistema los gestiona. En Linux, `init` los adopta.

? **`run()` vs `Popen()` — ¿cuándo usar cada uno?**

- `run()`: cuando necesitas el resultado y puedes esperar
- `Popen()`: cuando el proceso debe vivir en segundo plano mientras tú haces otras cosas

? **¿Puedo lanzar cualquier programa?** Sí, cualquier ejecutable. Pero el `PATH` debe incluirlo o debes dar la ruta completa.



Aprieta el lápiz

1. **PID del proceso:** Crea un programa que imprima su propio PID y el PID del proceso padre.
 2. **Lanzador de apps:** Usa `Popen` para abrir 3 programas distintos (bloc de notas, calculadora, navegador).
 3. **Comando con timeout:** Lanza `ping` con timeout de 3 segundos. Captura el error si excede.
 4. **Comunicación bidireccional:** Crea un proceso hijo que lea de `stdin` y devuelva el texto en mayúsculas.
 5. **Caza de procesos:** Lanza 5 procesos `notepad` y luego mata todos menos uno.
-

RAs cubiertos y criterios de evaluación

RA1 — Procesos

Criterio	Descripción	Cubierto
RA1a	Reconoce las características de los procesos	✓
RA1b	Distingue entre computación paralela y distribuida	✓
RA1c	Conoce los estados de un proceso	✓
RA1d	Identifica las diferencias clave entre proceso e hilo	→ T02
RA1e	Crea programas con procesos (subprocess)	✓
RA1f	Establece comunicación entre procesos	✓

RA1d se cubre en el **TEMA 02** junto con los hilos. RA1g (análisis ventajas procesos vs hilos) también en **TEMA 02**. RA1h (documentación) es transversal a todo el curso.

TEMA 02 — Hilos Fundamentos

TEMA 02 — Hilos Fundamentos (RA2)

“Un hilo es como una tarea dentro de una casa. Todos los hilos comparten la misma casa (memoria), pero cada uno hace su propia cosa.”

Índice

1. [¿Qué es un hilo?](#)
 2. [Hilos vs Procesos](#)
 3. [Crear y lanzar hilos](#)
 4. [join\(\) — esperar a que termine](#)
 5. [Hilos daemon](#)
 6. [Timer — ejecutar algo después de un tiempo](#)
 7. [El GIL y sus consecuencias](#)
 8. [Estados de un hilo](#)
 9. [Be the code, my friend, my friend — El hilo viajero](#)
 10. [Be the code, my friend, my friend — daemon en acción](#)
 11.  [El ring de los conceptos — Hilo normal vs Hilo daemon](#)
 12. [Preguntas tontas — Hilos](#)
 13.  [Aprieta el lápiz](#)
 14. [RAs cubiertos y criterios de evaluación](#)
-

¿Qué es un hilo?

Un **hilo** (thread) es la unidad más pequeña de ejecución. Un proceso puede tener múltiples hilos, todos compartiendo la misma memoria.

```
import threading

def saludar():
    print("¡Hola desde un hilo!")

hilo = threading.Thread(target=saludar)
hilo.start()
hilo.join()
```

Características

- Comparten memoria con otros hilos del mismo proceso
- Son más ligeros que los procesos (menos recursos al crearlos)
- Se comunican mediante variables compartidas (con cuidado)
- En Python, limitados por el **GIL** para código CPU-bound

Hilos vs Procesos

Característica	Proceso	Hilo
Memoria	Aislada (cada uno la suya)	Compartida (todos en la misma)
Creación	Lenta (el SO debe copiar recursos)	Rápida
Comunicación	Pipes, sockets, archivos	Variables globales
Aislamiento	Alto (uno no afecta a otro)	Bajo (uno puede romper a todos)
Coste	Alto	Bajo

“Los procesos son como casas separadas. Los hilos son como habitaciones de la misma casa.”

Crear y lanzar hilos

```
import threading

def trabajar(nombre, tarea):
    print(f"{nombre} empezando: {tarea}")
    # ... trabajo ...
    print(f"{nombre} terminó: {tarea}")

# Crear hilos con nombre
hilo_a = threading.Thread(target=trabajar, args=("Ana", "lavar platos"))
hilo_b = threading.Thread(target=trabajar, args=("Bob", "fregar suelo"))

# Lanzarlos
hilo_a.start()
hilo_b.start()

# Esperar
hilo_a.join()
hilo_b.join()
```

Propiedades de un hilo

```
hilo = threading.Thread(target=fn, name="hilo-1")
hilo.start()

print(hilo.name)          # "hilo-1"
print(hilo.ident)         # ID numérico
print(hilo.daemon)        # True/False
print(hilo.is_alive())    # True si sigue ejecutándose
```

join() — esperar a que termine

join() hace que el programa principal espere hasta que el hilo termine.

```
import threading, time

def trabajador(segundos):
    print(f"Trabajando durante {segundos}s...")
    time.sleep(segundos)
    print("Terminado")

h = threading.Thread(target=trabajador, args=(3,))
h.start()

print("Esperando al hilo...")
h.join() # Espera hasta que termine
print("El hilo terminó, continuamos")
```

Salida:

```
Trabajando durante 3s...
Esperando al hilo...
Terminado
El hilo terminó, continuamos
```

Sin `join()`, el programa principal seguiría y posiblemente terminaría antes que el hilo.

Hilos daemon

Un hilo **daemon** se ejecuta en segundo plano y **se mata automáticamente** cuando el programa principal termina.

```
import threading, time

def reloj():
    """Imprime la hora cada segundo (por siempre)"""
    while True:
        print(f"🕒 {time.strftime('%H:%M:%S')}")
        time.sleep(1)

hilo_reloj = threading.Thread(target=reloj, daemon=True)
hilo_reloj.start()

time.sleep(3) # El programa principal dura 3 segundos
print("Programa principal terminando...")
# Al terminar, el hilo daemon se mata solo
```

Salida:

```
🕒 14:35:22
🕒 14:35:23
🕒 14:35:24
Programa principal terminando...
```

Los hilos **no daemon** impiden que el programa termine. Los daemon se sacrifican para que el programa pueda salir.

Tipo	Comportamiento
daemon=False (defecto)	El programa espera a que termine
daemon=True	El programa lo mata al salir

Timer — ejecutar algo después de un tiempo


```
import threading

def aviso():
    print("🕒 ¡Tiempo cumplido!")

temporizador = threading.Timer(5.0, aviso)
temporizador.start()
print("Timer iniciado, 5 segundos...")

# temporizador.cancel() # Si queremos cancelar
```

Timer ejecuta la función **una sola vez** después del retardo. No se repite.

El GIL y sus consecuencias

GIL = Global Interpreter Lock. Es un candado interno de CPython que evita que dos hilos ejecuten bytecode Python a la vez.

¿Qué implica en la práctica?

```
import threading, time

# ⚡ CPU-bound: los hilos NO sirven de nada
def contar():
    total = 0
    for i in range(50_000_000):
        total += i

inicio = time.time()
hilos = [threading.Thread(target=contar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"4 hilos CPU: {time.time() - inicio:.2f}s")
# → ~5 segundos (igual que 1 hilo)
```

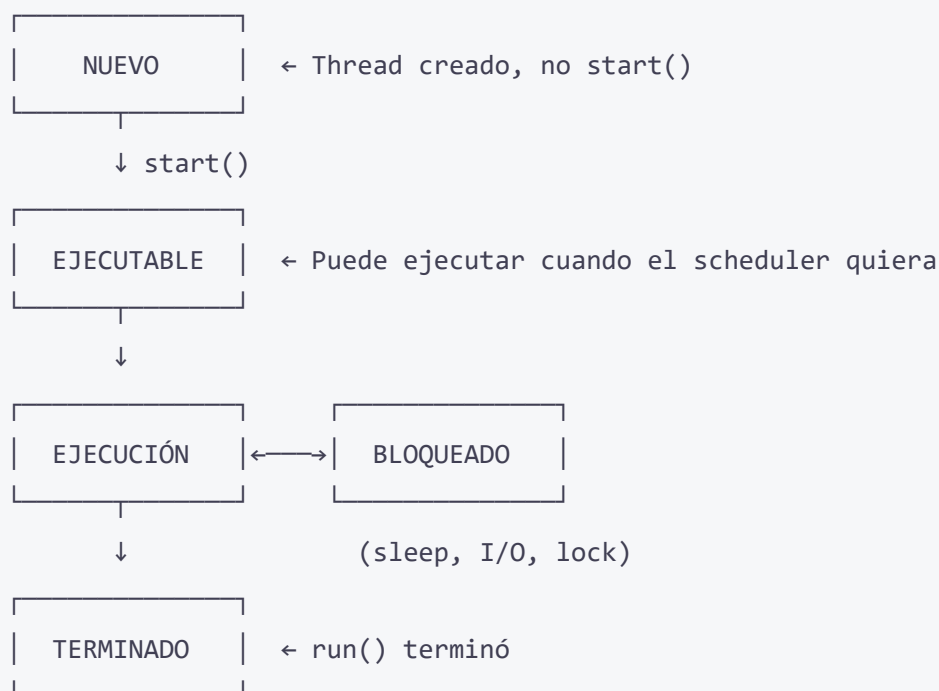
```
# 🌐 I/O-bound: los hilos SÍ aceleran
def esperar():
    time.sleep(2) # Simula descarga/lectura

inicio = time.time()
hilos = [threading.Thread(target=esperar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()
print(f"4 hilos I/O: {time.time() - inicio:.2f}s")
# → ~2 segundos (4 descargas en paralelo)
```

Tipo de tarea	¿Hilos ayudan?	Motivo
CPU-bound (calcular, procesar)	❌ No	El GIL solo deja ejecutar a uno
I/O-bound (esperar red, disco)	✅ Sí	El GIL se libera durante la espera

Para CPU-bound en Python, usa multiprocessing (varios procesos, cada uno con su propio GIL).

Estados de un hilo



Estado	Significado
NUEVO	El objeto Thread existe pero no se ha llamado a start()
EJECUTABLE	start() llamado. Puede ejecutar en cualquier momento
BLOQUEADO	Esperando (sleep, I/O, un lock)
TERMINADO	El método run() ha terminado

Be the code, my friend, my friend — El hilo viajero

“Sé el código y recorre su ejecución paso a paso. Dos hilos, dos viajeros.”

```
import threading, time

def viajero(nombre, paradas):
    for i in range(paradas):
        print(f"{nombre} 🚶 está en la parada {i+1}")
        time.sleep(0.5)
        print(f"{nombre} 🏠 ha llegado a su destino")

h1 = threading.Thread(target=viajero, args=("Ana", 3))
h2 = threading.Thread(target=viajero, args=("Bob", 2))

h1.start()
h2.start()
h1.join()
h2.join()
```

Traza (ejecución real):

```
▶ Ana (h1) empieza. Dice: "Ana 🚶 está en la parada 1"
▶ Bob (h2) empieza. Dice: "Bob 🚶 está en la parada 1"
⌚ Ana duerme 0.5s
⌚ Bob duerme 0.5s
▶ Bob se despierta. Dice: "Bob 🚶 está en la parada 2"
▶ Ana se despierta. Dice: "Ana 🚶 está en la parada 2"
⌚ Ana duerme 0.5s
⌚ Bob duerme 0.5s
▶ Bob se despierta. Dice: "Bob 🏁 ha llegado a su destino"
▶ Ana se despierta. Dice: "Ana 🚶 está en la parada 3"
⌚ Ana duerme 0.5s
▶ Ana se despierta. Dice: "Ana 🏁 ha llegado a su destino"
```

Fíjate cómo los print se entremezclan. El orden exacto lo decide el scheduler del sistema operativo. **No hay garantía de orden.**

Be the code, my friend, my friend — daemon en acción

```
import threading, time

def trabajador():
    for i in range(5):
        print(f"Trabajando... ({i+1}/5)")
        time.sleep(0.5)
    print("Trabajador terminó")

# SIN daemon — el programa espera
h1 = threading.Thread(target=trabajador)
h1.start()
h1.join()
print("Programa terminó (después del hilo)")
```

```
Trabajando... (1/5)
Trabajando... (2/5)
Trabajando... (3/5)
Trabajando... (4/5)
Trabajando... (5/5)
Trabajador terminó
Programa terminó (después del hilo)
```

```
# CON daemon – el hilo se mata al salir
h2 = threading.Thread(target=trabajador, daemon=True)
h2.start()
time.sleep(1.2)
print("Programa terminó (el hilo daemon muere conmigo)")
```

```
Trabajando... (1/5)
Trabajando... (2/5)
Trabajando... (3/5)
Programa terminó (el hilo daemon muere conmigo)
```

El daemon solo llegó a la iteración 3. El programa principal terminó y lo mató.

El ring de los conceptos — Hilo normal vs Hilo daemon

Hilo Normal: — Yo soy un hilo de verdad. El programa principal espera a que termine lo que tengo que hacer. Tengo responsabilidad.

Hilo Daemon: — ¡Qué aburrido! Yo soy libre. Mi única misión es servir en segundo plano. Cuando el programa principal termina, yo me muero con él, sin dramas.

Hilo Normal: — ¿Y si estás en medio de algo importante cuando el main termina? Pierdes datos, dejas cosas a medias...

Hilo Daemon: — Para eso existen los daemon bien hechos: tareas de monitoreo, limpieza, heartbeat... cosas que da igual si se cortan. Si quieres garantía de finalización, usas un hilo normal con `join()`.

Hilo Normal: — Cierto. Al final, cada uno tiene su sitio. Yo para tareas críticas, tú para servicios auxiliares.

Moraleja: Usa hilos **daemon** para servicios de fondo prescindibles. Usa hilos **normales** con `join()` para tareas que deben completarse sí o sí.

Preguntas tontas — Hilos

- ? **¿Un hilo puede crear otro hilo?** Sí. Un hilo puede lanzar otros hilos sin problema.
 - ? **¿Cuántos hilos puedo crear?** Hay límite práctico. En Windows, unos pocos miles. Cada hilo consume ~1MB de memoria virtual por su pila.
 - ? **¿Puedo matar un hilo desde fuera?** No limpiamente. No hay `hilo.kill()`. La forma correcta es usar una variable bandera que el hilo compruebe periódicamente.
 - ? **¿Qué pasa si no llamo a `join()`?** El hilo se ejecuta igual. Pero el programa principal no espera. Si es no-daemon, el programa no terminará hasta que el hilo termine.
 - ? **¿`sleep(0)` sirve para algo?** Sí, cede la CPU voluntariamente para que otro hilo pueda ejecutar. Es una “buena práctica” en hilos cooperativos.
 - ? **¿Los hilos tienen prioridad?** En Python, no hay prioridades nativas. El scheduler del SO decide. Puedes simular prioridades con lógica condicional, pero no es real.
-



Aprieta el lápiz

1. **Carrera de mensajes:** Crea 5 hilos que impriman su nombre 10 veces. Observa cómo se entremezclan.
2. **Temporizador daemon:** Crea un hilo daemon que imprima “tic” cada segundo. El programa principal espera 5 segundos y termina.
3. **Timer despertador:** Crea un Timer que imprima “¡Despierta!” a los 3 segundos.
4. **GIL test:** Compara el tiempo de 4 hilos vs 1 hilo haciendo una tarea CPU-bound (contar hasta 50M). Verifica que tardan lo mismo.
5. **I/O vs CPU:** Crea dos versiones de descarga simulada (`sleep 2s`) — una con 1 hilo y otra con 4. Mide la diferencia.

6. **Pool Puzzle:** Ordena las líneas para crear un programa que lance 2 hilos que saluden 3 veces cada uno.

RAs cubiertos y criterios de evaluación

RA2 — Hilos (parcial)

Criterio	Descripción	Cubierto
RA2a	Identifica la estructura de un hilo	
RA2b	Crea y lanza hilos con threading	
RA2e	Implementa esperas con join() y sleep()	
RA2f	Gestiona hilos daemon	
RA2h	Conoce el GIL y sus limitaciones	

Los criterios RA2c (Lock), RA2d (semáforos) y RA2g (condiciones de carrera) se cubren en el **TEMA 03 — Sincronización entre Hilos**.

TEMA 03 — Sincronización entre Hilos

TEMA 03 — Sincronización entre Hilos (RA2)

“Compartir memoria entre hilos sin sincronización es como compartir un cepillo de dientes. Alguien va a salir perdiendo.”

Índice

1. [El problema de la condición de carrera](#)
 2. [Lock — exclusión mutua](#)
 3. [Be the code, my friend, my friend — Contador con y sin Lock](#)
 4. [RLock — Lock reentrante](#)
 5. [Semaphore — acceso limitado](#)
 6. [Barrier — sincronización de grupo](#)
 7. [Condition — productor-consumidor](#)
 8.  [El ring de los conceptos — Lock vs Semáforo vs Barrera](#)
 9. [Be the code, my friend, my friend — Pool Puzzle de sincronización](#)
 10. [Preguntas tontas — Sincronización](#)
 11.  [Aprieta el lápiz](#)
 12. [RAs cubiertos y criterios de evaluación](#)
-

El problema de la condición de carrera

Dos hilos que incrementan una variable compartida sin sincronización:


```
import threading

contador = 0

def incrementar():
    global contador
    for _ in range(100_000):
        contador += 1 # ⚠ Esto NO es atómico

hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Esperado: 400.000 | Obtenido: {contador}")
# → 287.341, 312.045, 198.723... ¡nunca 400.000!
```

¿Por qué? contador += 1 en realidad hace 3 operaciones:

1. Leer contador de memoria
2. Sumar 1
3. Escribir el resultado

Si dos hilos leen el mismo valor antes de que ninguno haya escrito, ambos escriben el mismo resultado y se pierde un incremento.

Esto es una **condición de carrera** (race condition). Para evitarlo, necesitamos **sincronización**.

Lock — exclusión mutua

Lock garantiza que solo un hilo entre en una sección crítica a la vez.

```

import threading

contador = 0
lock = threading.Lock()

def incrementar():
    global contador
    for _ in range(100_000):
        with lock:          # 🤝 Solo un hilo aquí dentro
            contador += 1

hilos = [threading.Thread(target=incrementar) for _ in range(4)]
for h in hilos: h.start()
for h in hilos: h.join()

print(f"Con Lock: {contador}") # ✅ 400.000

```

Métodos de Lock

Método	Qué hace
<code>lock.acquire()</code>	Bloquea. Si otro hilo lo tiene, espera
<code>lock.release()</code>	Libera. Otro hilo puede entrar
<code>lock.locked()</code>	Devuelve True si está bloqueado

Siempre usa `with lock:`. Si usas `acquire()` manual y olvidas `release()`, creas un **deadlock**.

Be the code, my friend, my friend — Contador con y sin Lock

Sin Lock — el caos:

```
Hilo-A: lee contador = 0
Hilo-B: lee contador = 0      ← ¡mismo valor!
Hilo-A: escribe contador = 1
Hilo-B: escribe contador = 1  ← ¡pisó el incremento de A!
Hilo-A: lee contador = 1
Hilo-A: escribe contador = 2
Hilo-B: lee contador = 1      ← ¡leyó valor viejo!
```

Con Lock — el orden:

```
Hilo-A: acquire() → DENTRO
Hilo-A: lee contador = 0
Hilo-A: escribe contador = 1
Hilo-A: release() → FUERA

Hilo-B: acquire() → DENTRO      ← esperó hasta que A soltó
Hilo-B: lee contador = 1        ← valor correcto
Hilo-B: escribe contador = 2
Hilo-B: release() → FUERA

Hilo-C: acquire() → DENTRO
Hilo-C: lee contador = 2        ← valor correcto
...
```

RLock — Lock reentrante

Un Lock normal no permite que el mismo hilo lo adquiriera dos veces. Un RLock sí.

```
import threading

# Lock normal – DEADLOCK si el mismo hilo intenta adquirirlo dos veces
lock = threading.Lock()
lock.acquire()
lock.acquire() # ⚠ El hilo se espera a sí mismo → DEADLOCK

# RLock – el mismo hilo puede adquirirlo varias veces
rlock = threading.RLock()
rlock.acquire() # ok
rlock.acquire() # ok (mismo hilo)
rlock.release()
rlock.release()
```

Útil en funciones que se llaman entre sí recursivamente.

Semaphore — acceso limitado

Un Semaphore permite que hasta **N hilos** accedan al recurso a la vez.

```
import threading, time

semaforo = threading.Semaphore(2) # Máximo 2 hilos dentro

def entrar(id):
    print(f"Hilo-{id} esperando...")
    with semaforo:
        print(f" → Hilo-{id} DENTRO")
        time.sleep(2)
        print(f" ← Hilo-{id} SALE")

hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(5)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida (salen de 2 en 2):

```
Hilo-0 esperando...
Hilo-1 esperando...
Hilo-2 esperando...
Hilo-3 esperando...
Hilo-4 esperando...
  → Hilo-0 DENTRO
  → Hilo-1 DENTRO
  ← Hilo-0 SALE
  ← Hilo-1 SALE
  → Hilo-2 DENTRO
  → Hilo-3 DENTRO
  ← Hilo-2 SALE
  ← Hilo-3 SALE
  → Hilo-4 DENTRO
  ← Hilo-4 SALE
```

Útil para: limitar conexiones a una BD, descargas simultáneas, acceso a una API con rate limit.

Barrier — sincronización de grupo

Una Barrier obliga a todos los hilos a esperar hasta que el último llegue. Entonces todos continúan a la vez.

```
import threading, time

barrera = threading.Barrier(3) # 3 hilos deben llegar

def corredor(id):
    print(f"Corredor-{id} preparándose...")
    time.sleep(id * 0.5) # Cada uno tarda distinto
    print(f" → Corredor-{id} en la salida")
    barrera.wait() # Espera a los demás
    print(f" 🏁 Corredor-{id} SALIÓ!")

hilos = [threading.Thread(target=corredor, args=(i,)) for i in range(3)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Salida:

```
Corredor-0 preparándose...
  → Corredor-0 en la salida
Corredor-1 preparándose...
  → Corredor-1 en la salida
Corredor-2 preparándose...
  → Corredor-2 en la salida
  ❏ Corredor-0 SALIÓ!
  ❏ Corredor-1 SALIÓ!
  ❏ Corredor-2 SALIÓ!
```

Los 3 salen exactamente a la vez. Barrier es perfecto para sincronizar **fases** de un trabajo en paralelo.

Condition — productor-consumidor

Condition permite que un hilo espere hasta que otro le notifique que algo ocurrió.

```

import threading, time, random

cola = []
condition = threading.Condition()

def productor():
    for i in range(5):
        with condition:
            item = random.randint(1, 100)
            cola.append(item)
            print(f"📦 Producido: {item}")
            condition.notify() # Avisar al consumidor
            time.sleep(random.random())

def consumidor():
    for _ in range(5):
        with condition:
            while not cola:
                condition.wait() # Espera a que haya algo
            item = cola.pop(0)
            print(f"🥤 Consumido: {item}")

h_prod = threading.Thread(target=productor)
h_cons = threading.Thread(target=consumidor)
h_prod.start()
h_cons.start()
h_prod.join()
h_cons.join()

```

Método de Condition	Qué hace
<code>wait()</code>	Libera el lock y espera una notificación
<code>notify()</code>	Despierta a un hilo que está esperando
<code>notify_all()</code>	Despierta a todos los hilos que esperan

Siempre usa `while` en lugar de `if` para comprobar la condición después de `wait()` — pueden darse **falsas activaciones** (spurious wakeups).



El ring de los conceptos — Lock vs Semáforo vs Barrera

Lock: “Soy el portero. Solo dejo pasar a **uno cada vez**. Los demás esperan fuera.”

Semáforo(3): “Soy el guardia de una sala con **3 sillas**. Cuando una se libera, entra el siguiente.”

Barrera(3): “Soy el juez de salida. **No permito que nadie corra** hasta que los 3 estén en la línea.”

Lock: “Para exclusión mutua, no hay mejor que yo. Dos hilos no pueden tocarme el recurso a la vez.”

Semáforo: “Para **limitar acceso concurrente**. Como un aforo máximo en un local.”

Barrera: “Para sincronizar **fases de un trabajo**. Todos completan la fase 1, luego todos empiezan la fase 2.”

Be the code, my friend, my friend — Pool Puzzle de sincronización

Instrucciones: Ordena estas líneas para que 3 hilos incrementen un contador compartido de forma segura.

```
a. lock = threading.Lock()
b. for h in hilos: h.start()
c. contador += 1
d. hilos = []
e. import threading
f. for _ in range(3):
g. with lock:
h. def incrementar():
i.     hilos.append(threading.Thread(target=incrementar))
j. for h in hilos: h.join()
k. global contador
l. contador = 0
m. print(contador)
```

› Solución

Preguntas tontas — Sincronización

? **¿Qué es un deadlock?** Dos o más hilos esperándose mutuamente para siempre. Ejemplo: Hilo-A tiene Lock-1 y espera Lock-2; Hilo-B tiene Lock-2 y espera Lock-1.

? **¿Cómo evito deadlocks?**

1. Adquirir los locks siempre en el **mismo orden**
2. Usar `with lock:` (nunca olvidar `release()`)
3. Usar `RLock` si el mismo hilo necesita adquirirlo varias veces

? **¿Puedo tener más de un Lock?** Sí. Pero cada Lock añade riesgo de deadlock. Sé disciplinado con el orden de adquisición.

? **¿Qué es más rápido, Lock o Semaphore?** Lock es más simple. Semaphore tiene contador interno. Para exclusión mutua, Lock gana en velocidad.

? **¿Y si necesito sincronización entre procesos en vez de entre hilos?** Usa `multiprocessing.Lock`, `multiprocessing.Semaphore`, etc. Son equivalentes pero para procesos.



Aprieta el lápiz

1. **Condición de carrera:** Crea 2 hilos que incrementen un contador 1M veces cada uno. Compara con y sin Lock.
 2. **Semáforo descargas:** Crea 10 hilos que simulen descargas (`sleep 2s`) usando un `Semáforo(3)`. Mide el tiempo total.
 3. **Barrera de fases:** Crea 4 hilos que trabajen en 2 fases. La fase 2 no empieza hasta que todos terminan la fase 1.
 4. **Productor-Consumidor:** 1 productor crea números aleatorios cada 0.5s; 2 consumidores los procesan. Usa `Condition`.
 5. **Deadlock provocado:** Crea intencionadamente un deadlock con 2 hilos y 2 locks. Luego arréglalo.
-

RAs cubiertos y criterios de evaluación

RA2 — Hilos (sincronización)

Criterio	Descripción	Cubierto
RA2c	Sincroniza hilos con Lock	✓
RA2d	Usa semáforos para acceso controlado	✓
RA2g	Evita condiciones de carrera	✓
(RA2a-b, RA2e-f, RA2h)	Cubiertos en el TEMA 02	←

TEMA 04 — Sockets TCP

TEMA 04 — Sockets TCP (RA3)

“Un socket TCP es como una llamada telefónica: marcas, esperas a que contesten, habláis y colgáis.”

Índice

1. [¿Qué es un socket?](#)
2. [TCP — orientado a conexión](#)
3. [Servidor TCP — ciclo de vida](#)
4. [Cliente TCP — ciclo de vida](#)
5. [Be the code, my friend, my friend — Mano a mano TCP](#)
6. [SO_REUSEADDR — no más “Address already in use”](#)
7. [Errores comunes y cómo manejarlos](#)
8. [Non-blocking y timeouts](#)
9.  [El ring de los conceptos — Servidor vs Cliente](#)
10. [Preguntas tontas — Sockets TCP](#)
11.  [Aprieta el lápiz](#)
12. [RAs cubiertos y criterios de evaluación](#)

¿Qué es un socket?

Un **socket** es el punto final de una conexión de red. Es la interfaz que tu programa usa para enviar y recibir datos a través de la red.

```
import socket

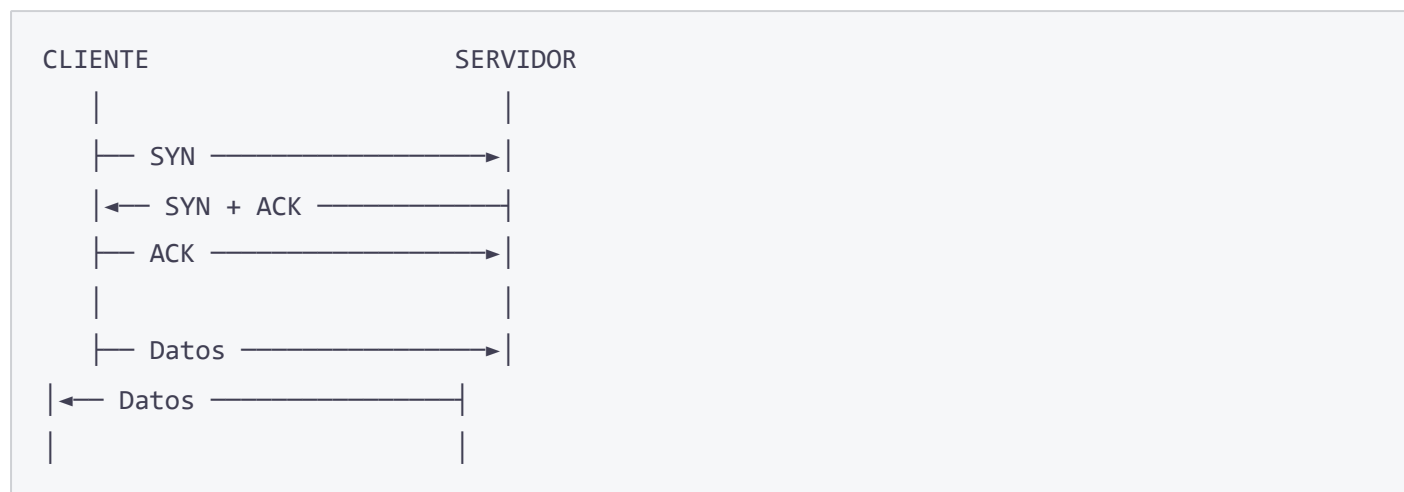
# Crear un socket TCP
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Parámetro	Significado	Valores
AF_INET	Familia de direcciones	IPv4
AF_INET6	IPv6	
SOCK_STREAM	TCP (orientado a conexión)	
SOCK_DGRAM	UDP (sin conexión)	

TCP — orientado a conexión

TCP garantiza que los datos lleguen **en orden** y **sin pérdidas**. A cambio, es un poco más lento que UDP.

Three-way handshake (establecer conexión)



Servidor TCP — ciclo de vida

```
socket() → bind() → listen() → accept() → recv()/send() → close()
```

```
import socket

HOST = "127.0.0.1" # localhost
PORT = 5000

# 1. Crear socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as servidor:

    # 2. Asignar dirección y puerto
    servidor.bind((HOST, PORT))

    # 3. Escuchar conexiones entrantes
    servidor.listen()
    print(f"Servidor escuchando en {HOST}:{PORT}")

    # 4. Aceptar una conexión (BLOQUEANTE)
    conexion, direccion = servidor.accept()
    print(f"Conectado con {direccion}")

    with conexion:
        # 5. Recibir datos
        datos = conexion.recv(1024)
        print(f"Recibido: {datos.decode()}")

        # 6. Enviar respuesta
        conexion.sendall(b"Recibido: " + datos)

    # 7. Se cierra solo al salir del with
```

Cliente TCP — ciclo de vida

```
socket() → connect() → send()/recv() → close()
```

```
import socket

HOST = "127.0.0.1"
PORT = 5000

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as cliente:
    cliente.connect((HOST, PORT))
    cliente.sendall(b"Hola servidor!")
    respuesta = cliente.recv(1024)
    print(f"Respuesta: {respuesta.decode()}")
```

Be the code, my friend, my friend — Mano a mano TCP

“Sé el servidor y el cliente. Cada paso, cada byte.”

● SERVIDOR

1. `socket(AF_INET, SOCK_STREAM) → srv`
2. `bind(("127.0.0.1", 5000))`
3. `listen()`
4. `accept() → espera...` ⌚

● CLIENTE

1. `socket(AF_INET, SOCK_STREAM) → cli`
2. `connect(("127.0.0.1", 5000))`
→ Three-way handshake TCP

● SERVIDOR

4. `accept()` devuelve `(conn, ("127.0.0.1", 54321))`
5. Esperando datos... ⌚

● CLIENTE

3. `sendall(b"Hola servidor!")`
4. Esperando respuesta... ⌚

● SERVIDOR

5. `recv(1024) → b"Hola servidor!"`
6. `print("Recibido: Hola servidor!")`
7. `sendall(b"Recibido: Hola servidor!")`

● CLIENTE

4. `recv(1024) → b"Recibido: Hola servidor!"`
5. `print("Respuesta: Recibido: Hola servidor!")`
6. `close()`

● SERVIDOR

8. `close()`

`accept()` y `recv()` son **bloqueantes**: el programa se queda esperando hasta que algo ocurra.

SO_REUSEADDR — no más “Address already in use”

Si matas un servidor y lo reinicias rápido, el SO puede decir que la dirección ya está en uso. La opción `SO_REUSEADDR` lo evita.

```
import socket

servidor = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
servidor.bind(("127.0.0.1", 5000))
servidor.listen()
```

Pon esto siempre en tus servidores. Te ahorrarás minutos de depuración.

Errores comunes y cómo manejarlos

```
import socket, time

def conectar_seguro(host, port, reintentos=3):
    for intento in range(reintentos):
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                s.settimeout(5)
                s.connect((host, port))
                s.sendall(b"test")
                return s.recv(1024)
        except socket.timeout:
            print(f"⌚ Timeout (intento {intento+1})")
        except ConnectionRefusedError:
            print(f"🚫 Conexión rechazada – ¿el servidor está encendido?")
            time.sleep(1)
        except ConnectionResetError:
            print(f"💥 El servidor cerró la conexión abruptamente")
        except OSError as e:
            print(f"🚧 Error de red: {e}")
    return None
```

Excepción	Cuándo ocurre
<code>socket.timeout</code>	La operación excede el tiempo límite
<code>ConnectionRefusedError</code>	No hay nadie escuchando en ese puerto
<code>ConnectionResetError</code>	El otro lado cerró de golpe

Excepción	Cuándo ocurre
OSError	Red caída, DNS no resuelve, etc.

Non-blocking y timeouts

```
import socket, select

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Opción 1: timeout fijo
s.settimeout(5.0)

# Opción 2: no bloqueante (lanza excepción si no hay datos)
s.setblocking(False)

# Opción 3: select (esperar con timeout en múltiples sockets)
lectura, _, _ = select.select([s], [], [], 1.0)
if lectura:
    datos = s.recv(1024)
```

`select.select()` es la solución para esperar en varios sockets a la vez sin hilos.

Pool Puzzle — El ciclo de vida del socket

Las líneas de abajo están desordenadas. ¿Puedes ordenarlas para que formen un servidor TCP funcional?

```
a) with socket.socket() as srv:
b)     srv.listen()
c)     conn, addr = srv.accept()
d)     import socket
e)         conn.sendall(b"Bienvenido\n")
f)     srv.bind(("127.0.0.1", 5000))
g)     with conn:
h)         datos = conn.recv(1024)
```

El ring de los conceptos — Servidor vs Cliente

Servidor: — Yo soy el que hace todo el trabajo. Escucho, acepto, atiendo... todo el peso recae sobre mí.

Cliente: — ¿Trabajo dices? Yo soy el que inicia todo. Si no fuera por mí, tú estarías ahí escuchando en el puerto para siempre, como una planta.

Servidor: — Pero tengo que gestionar múltiples conexiones, mantener el estado, no caerme... Tú solo te conectas, mandas algo y te vas.

Cliente: — Y también gestiono errores: ¿y si el servidor no está? ¿y si hay timeout? ¿y si la red falla? No es tan sencillo.

Servidor: — Vale, vale. En realidad somos un equipo. Sin servidor no hay servicio, pero sin cliente no hay razón para existir.

Moraleja: Servidor y cliente son dos caras de la misma moneda. El protocolo (quién envía qué y cuándo) es el verdadero protagonista.

Preguntas tontas — Sockets TCP

? ¿Qué pasa si el servidor no llama a `listen()`? El cliente recibe `ConnectionRefusedError`. Sin `listen()`, no hay puerto abierto.

? ¿Y si no llamo a `bind()`? Para el servidor es obligatorio (necesita un puerto fijo). Para el cliente, el SO asigna uno automáticamente.

? ¿`recv(1024)` significa que solo puedo recibir 1024 bytes? No, es el tamaño máximo del buffer. Si el mensaje es más grande, necesitas varios `recv()` y concatenar. Tú tienes que implementar el protocolo de aplicación para saber cuándo has recibido el mensaje completo.

? ¿Puedo tener más de un cliente conectado a la vez? Con este código básico, no. Solo acepta un cliente cada vez. Para múltiples clientes, mira el **TEMA 10 — Servidores Concurrentes**.

? ¿Qué es “127.0.0.1”? Es **localhost** — tu propia máquina. Perfecto para pruebas. Para que otros se conecten, usa tu IP real (ej: 192.168.1.x).

? ¿Y si un cliente envía datos muy seguido? TCP los encola. El servidor los recibe en orden. Pero si el cliente envía más rápido de lo que el servidor lee, el buffer se llena y el cliente se bloquea.

Aprieta el lápiz

1. **Eco server**: Crea un servidor que devuelva exactamente lo que recibe.
2. **Contador de letras**: El cliente envía una frase, el servidor responde con la cantidad de letras.
3. **Servidor hora**: El cliente se conecta y el servidor le devuelve la hora actual.
4. **Cliente con timeout**: Crea un cliente que intente conectar, y si no hay respuesta en 3s, muestre “Servidor no disponible”.

RAs cubiertos y criterios de evaluación

RA3 — Sockets (parcial: TCP)

Criterio	Descripción	Cubierto
RA3a	Modelo de capas de red (TCP/IP)	✓
RA3c	Crea servidores TCP	✓
RA3d	Crea clientes TCP	✓
RA3f	Gestiona errores de red	✓
RA3g	Configura opciones de socket (SO_REUSEADDR, non-blocking)	✓

RA3b (UDP), RA3e (UDP servidor/cliente) y RA3h (protocolos HTTP/NTP) se cubren en el TEMA 05.

TEMA 05 — Sockets UDP y Protocolos

TEMA 05 — Sockets UDP y Protocolos (RA3)

“UDP es como lanzar un avión de papel. TCP es como enviar una carta certificada. Cada uno tiene su momento.”

Índice

1. [UDP — sin conexión](#)
2. [Servidor UDP](#)
3. [Cliente UDP](#)
4.  [El ring de los conceptos — TCP vs UDP](#)
5. [HTTP desde cero — el protocolo que mueve la web](#)
6. [Be the code, my friend, my friend — Cliente HTTP manual](#)
7. [NTP — ¿qué hora es en Internet?](#)
8. [Preguntas tontas — UDP y Protocolos](#)
9.  [Aprieta el lápiz](#)
10. [RAs cubiertos y criterios de evaluación](#)

UDP — sin conexión

UDP **no establece conexión**. Mandas el mensaje y rezas porque llegue. No hay garantía de entrega ni de orden.

```
import socket

# Socket UDP
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Característica	TCP	UDP
Conexión	Sí (handshake)	No
Entrega garantizada	Sí	No

Característica	TCP	UDP
Orden	Sí	No
Velocidad	Más lento	Más rápido
Uso típico	Web, email, FTP	Streaming, juegos, DNS

Servidor UDP

Sin `accept()`, sin `listen()`. Solo `recvfrom()`.

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as servidor:
    servidor.bind((HOST, PORT))
    print(f"Servidor UDP en {HOST}:{PORT}")

    # Recibir datagrama (recvfrom devuelve datos + dirección)
    datos, direccion = servidor.recvfrom(1024)
    print(f"Recibido de {direccion}: {datos.decode()}")

    # Responder
    servidor.sendto(b"Recibido!", direccion)
```

Cliente UDP

Sin `connect()`. `sendto()` en vez de `send()`.

```
import socket

HOST = "127.0.0.1"
PORT = 5001

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cliente:
    cliente.sendto(b"Hola UDP!", (HOST, PORT))
    datos, _ = cliente.recvfrom(1024)
    print(f"Respuesta: {datos.decode()}")
```

En UDP, cada `sendto()` es un datagrama independiente. Pueden llegar desordenados, duplicados o no llegar.



El ring de los conceptos — TCP vs UDP

TCP: “Yo soy el mensajero certificado. Entrego cada carta, en orden, y si se pierde, la reenvío. Pero cuesta más.”

UDP: “Yo soy el lanzador de aviones de papel. Mando y olvido. Si no llega, pues no llega. Pero lanzo 100 en el tiempo que tú preparas uno.”

TCP: “Mis casos de uso: web (HTTP), correo (SMTP), transferencia de archivos (FTP). Todo lo que necesite fiabilidad.”

UDP: “Mis casos de uso: videollamadas (Zoom), streaming (Twitch), juegos online (Fortnite), DNS. Prefiero velocidad antes que fiabilidad.”

TCP: “Tengo control de congestión, retransmisión, checksums...”

UDP: “Yo tengo... velocidad. Y puedo añadir fiabilidad en la capa de aplicación si quiero (QUIC, por ejemplo).”

TCP: “Eres un temerario.”

UDP: “Y tú un pesado. Por eso nos complementamos.”

HTTP desde cero — el protocolo que mueve la web

HTTP es un protocolo de **texto** sobre TCP. Un cliente envía una petición y el servidor responde.

Petición HTTP

```
GET /ruta HTTP/1.1\r\n
Host: ejemplo.com\r\n
User-Agent: MiNavegador\r\n
Accept: text/html\r\n
\r\n
```

Respuesta HTTP

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 1234\r\n
\r\n
<html>...cuerpo...</html>
```

Be the code, my friend, my friend — Cliente HTTP manual

“Sé el navegador. Olvida requests. Abre un socket y habla HTTP como en los viejos tiempos.”

```

import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    # Conectar al servidor web
    s.connect(("www.example.com", 80))

    # Enviar petición HTTP/1.1
    petition = (
        "GET / HTTP/1.1\r\n"
        "Host: www.example.com\r\n"
        "Connection: close\r\n"
        "\r\n"
    )
    s.sendall(petition.encode())

    # Recibir respuesta
    respuesta = b""
    while True:
        datos = s.recv(4096)
        if not datos:
            break
        respuesta += datos

    # Mostrar los primeros 500 caracteres
    print(respuesta.decode()[:500])

```

Traza:

1. `socket()` → crea socket TCP
2. `connect("www.example.com", 80)` → three-way handshake
3. Construye string HTTP: `"GET / HTTP/1.1\r\nHost: www.example.com\r\n..."`
4. `sendall` → envía bytes
5. Bucle `recv` → va recibiendo trozos de la respuesta
6. El servidor cierra conexión → `recv` devuelve `b""`
7. Decodifica bytes a string
8. Muestra `"<!doctype html>..."`  ¡es el HTML de la web!

NTP — ¿qué hora es en Internet?

NTP (Network Time Protocol) usa UDP para sincronizar relojes.

```
import socket, struct, time

def hora_ntp():
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(5)
        # Paquete NTP: 48 bytes, modo cliente
        paquete = b'\x1b' + 47 * b'\0'
        s.sendto(paquete, ("pool.ntp.org", 123))
        datos, _ = s.recvfrom(1024)

        # El timestamp está en los bytes 40-43
        t = struct.unpack('!I', datos[40:44])[0]
        # Ajustar época NTP (1900) a Unix (1970)
        return t - 2208988800

hora = hora_ntp()
print(f"Hora NTP oficial: {time.ctime(hora)}")
```

Así es como tu ordenador sabe la hora exacta sin tener un reloj atómico.

Preguntas tontas — UDP y Protocolos

? ¿Cuándo usar UDP en vez de TCP? Cuando la velocidad importa más que la fiabilidad: streaming, juegos, VoIP, DNS. Perder un frame de video es mejor que esperar a que se reenvíe.

? ¿UDP puede perder datos? Sí. No hay confirmación de recepción. Si pierdes un paquete, se pierde para siempre.

? ¿HTTP siempre usa TCP? Sí, HTTP/1.1 y HTTP/2 usan TCP. **HTTP/3** usa QUIC, que va sobre UDP (¡la vuelta a la tortilla!).

? ¿NTP usa UDP? ¿No es importante que llegue la hora exacta? Sí, NTP usa UDP. Pero manda muchas peticiones y calcula estadísticamente la hora correcta. Si un paquete se pierde, no pasa nada: el próximo valdrá.



Aprieta el lápiz

1. **Servidor UDP eco:** Crea un servidor UDP que devuelva al cliente lo mismo que recibe.
2. **Cliente HTTP manual:** Conéctate a `httpbin.org` y haz un GET a `/ip` para ver tu IP pública.
3. **Mini servidor web:** Crea un servidor TCP que responda con HTML básico (“

Hola

”) a cualquier petición.

4. **Comparativa velocidad:** Mide cuánto tarda TCP vs UDP en enviar 100 mensajes pequeños.

RAs cubiertos y criterios de evaluación

RA3 — Sockets (completo)

Criterio	Descripción	Cubierto
RA3a	Modelo de capas de red (TCP/IP)	✓
RA3b	Identifica tipos de sockets (TCP/UDP)	✓
RA3e	Implementa servidores y clientes UDP	✓
RA3h	Implementa protocolos de aplicación (HTTP, NTP)	✓

RA3c (servidor TCP), RA3d (cliente TCP), RA3f (errores) y RA3g (opciones) se cubren en el **TEMA 04**.

TEMA 06 — APIs REST y HTTP

TEMA 06 — APIs REST y HTTP (RA4a-b)

“Una API REST es como un camarero: tú le dices lo que quieres (GET/POST), y él te lo trae (JSON). No tienes que saber cómo lo cocina la cocina.”

Índice

1. [¿Qué es una API?](#)
 2. [REST — el estándar de las APIs modernas](#)
 3. [HTTP — los verbos](#)
 4. [Códigos de estado — ¿salió bien o mal?](#)
 5. [La librería requests](#)
 6. [JSON — el idioma de las APIs](#)
 7. [Cabeceras y parámetros](#)
 8. [Be the code, my friend, my friend — Petición paso a paso](#)
 9.  [El ring de los conceptos — GET vs POST vs PUT vs DELETE](#)
 10. [Preguntas tontas — APIs y REST](#)
 11.  [Aprieta el lápiz](#)
 12. [RAs cubiertos y criterios de evaluación](#)
-

¿Qué es una API?

API = Application Programming Interface. Un conjunto de reglas para que dos programas se comuniquen.

```
# Sin API — accedes directamente a la BD (malo)
import sqlite3
conn = sqlite3.connect("usuarios.db")
conn.execute("SELECT * FROM usuarios")

# Con API — pides datos a un servicio
import requests
respuesta = requests.get("https://miapi.com/usuarios")
datos = respuesta.json()
```

Las APIs ponen una capa de abstracción: no necesitas saber cómo está implementado el sistema por dentro.

REST — el estándar de las APIs modernas

REST (Representational State Transfer) se basa en:

- **Recursos:** cada entidad tiene una URL (/usuarios, /productos/123)
- **Verbos HTTP:** GET (leer), POST (crear), PUT (actualizar), DELETE (borrar)
- **Sin estado:** cada petición contiene todo lo necesario
- **JSON:** formato de intercambio

GET	/usuarios	→ Listar usuarios
GET	/usuarios/5	→ Obtener usuario 5
POST	/usuarios	→ Crear usuario
PUT	/usuarios/5	→ Actualizar usuario 5
DELETE	/usuarios/5	→ Borrar usuario 5

HTTP — los verbos

Método	CRUD	Uso	Cuerpo
GET	Read	Obtener datos	 No
POST	Create	Crear recurso	 Sí
PUT	Update	Reemplazar completo	 Sí

Método	CRUD	Uso	Cuerpo
PATCH	Update	Actualizar parcial	✅ Sí
DELETE	Delete	Borrar recurso	❌ No

```
import requests

# POST – crear un recurso
nuevo_usuario = {"nombre": "Ana", "email": "ana@x.com"}
resp = requests.post("https://miapi.com/usuarios", json=nuevo_usuario)
print(resp.status_code) # 201 Created

# DELETE – borrar
resp = requests.delete("https://miapi.com/usuarios/5")
print(resp.status_code) # 204 No Content
```

Códigos de estado — ¿salió bien o mal?

Código	Significado	Situación típica
200	OK	GET exitoso
201	Created	POST exitoso (recurso creado)
204	No Content	DELETE exitoso
301	Moved Permanently	La URL cambió
400	Bad Request	Enviaste datos incorrectos
401	Unauthorized	Falta autenticación
403	Forbidden	No tienes permiso
404	Not Found	El recurso no existe
429	Too Many Requests	Te pasaste del límite
500	Internal Server Error	Error del servidor

Código	Significado	Situación típica
503	Service Unavailable	El servicio está caído

Consejo: los códigos 2xx son éxito, 4xx son error TUYO, 5xx son error DEL SERVIDOR.

```
resp = requests.get("https://api.github.com/users/python")
if resp.status_code == 200:
    print("✅ Todo bien:", resp.json()["login"])
elif resp.status_code == 404:
    print("❌ Usuario no encontrado")
else:
    print(f"⚠️ Código inesperado: {resp.status_code}")
```

La librería requests

requests es la librería estándar de facto para hacer peticiones HTTP en Python.

```
import requests

# GET básico
resp = requests.get("https://api.github.com")
print(resp.status_code)      # 200
print(resp.headers)          # Cabeceras de respuesta
print(resp.elapsed)          # Tiempo que tardó

# POST con JSON
resp = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json={"title": "foo", "body": "bar", "userId": 1}
)
print(resp.json())           # La respuesta parseada como dict
```

Propiedades de una respuesta

Propiedad	Qué devuelve
resp.status_code	Código HTTP (int)

Propiedad	Qué devuelve
<code>resp.ok</code>	True si 200-399
<code>resp.text</code>	Cuerpo como string
<code>resp.json()</code>	Cuerpo como dict (¡lanza excepción si no es JSON!)
<code>resp.headers</code>	Dict con cabeceras
<code>resp.elapsed</code>	Objeto timedelta

JSON — el idioma de las APIs

JSON = JavaScript Object Notation. Es el formato de intercambio de datos más usado en APIs.

```
{
  "nombre": "Ana",
  "edad": 25,
  "email": "ana@x.com",
  "activo": true,
  "intereses": ["Python", "redes", "cripto"]
}
```

En Python, JSON se convierte automáticamente a diccionarios y listas:

```
import requests

resp = requests.get("https://api.github.com/users/python")
datos = resp.json()

print(datos["login"])          # "python"
print(datos["public_repos"])   # 42
print(datos["avatar_url"])     # "https://..."
```

Cabeceras y parámetros

Parámetros de consulta (query params)

```
params = {"q": "python", "page": 1, "per_page": 10}
resp = requests.get("https://api.github.com/search/repositories", params=params)
# URL final: https://api.github.com/search/repositories?q=python&page=1&per_page=10
```

Cabeceras personalizadas

```
cabeceras = {
    "Authorization": "Bearer mi-token-secreto",
    "Accept": "application/json",
    "User-Agent": "MiApp/1.0"
}
resp = requests.get("https://api.github.com/user", headers=cabeceras)
```

Be the code, my friend, my friend — Petición paso a paso

“Sé la librería requests y recorre cada milisegundo de una petición GET.”

```
import requests

# 1. Construimos la URL y parámetros
url = "https://api.github.com/users/python"

# 2. Llamamos a requests.get()
respuesta = requests.get(url)

# 3. Procesamos la respuesta
datos = respuesta.json()
print(f"Usuario: {datos['login']}")
print(f"Repos: {datos['public_repos']}")
```

Traza interna:

1. `requests.get("https://api.github.com/users/python")`
2. `requests` construye la URL:
 - scheme: `https`
 - host: `api.github.com`
 - path: `/users/python`
3. Abre conexión TCP a `api.github.com:443` (HTTPS)
 - DNS lookup: `api.github.com` → `140.82.121.5`
 - Three-way handshake TCP
 - Handshake TLS (cifrado)
4. Envía petición HTTP:
`GET /users/python HTTP/1.1`
`Host: api.github.com`
`User-Agent: python-requests/2.31.0`
`Accept-Encoding: gzip, deflate`
`Accept: */*`
5. ⏰ Espera respuesta (~150ms)
6. Recibe respuesta:
`HTTP/1.1 200 OK`
`Content-Type: application/json`
...
7. `requests.parsea` el JSON → dict de Python
8. Extraemos `datos['login']` → `"python"`

Pool Puzzle — Petición API con errores

Estas líneas hacen una petición a una API y manejan errores. ¿En qué orden van?

```
a)     if respuesta.status_code == 200:
b) import requests
c)         print("Error:", respuesta.status_code)
d) respuesta = requests.get("https://api.github.com/users/python")
e)         datos = respuesta.json()
f)     else:
g)         print(datos["login"])
h) try:
```

>  Solución

El ring de los conceptos — GET vs POST vs PUT vs DELETE

GET: — Yo soy el más usado. Solo pido información, no cambio nada. Soy seguro, idempotente y cacheable.

POST: — Pues yo soy el que crea cosas. Sin mí no habría nuevos recursos. Eso sí, no soy idempotente: si llamas dos veces, creo dos veces.

PUT: — Idempotente, como GET, pero yo actualizo. Mando el recurso completo y reemplazo lo que haya.

DELETE: — Y yo elimino. También idempotente: da igual cuántas veces lo llames, el recurso ya no está.

GET: — Oye, ¿y PATCH? Siempre se nos olvida...

POST: — Ese es el primo moderno que actualiza solo un campo. Pero mejor no liarnos, que ya somos suficientes.

Moraleja: Cada verbo HTTP tiene un propósito: GET (leer), POST (crear), PUT (reemplazar), DELETE (borrar). Elegir el correcto es hacer bien REST.

Preguntas tontas — APIs y REST

? ¿Necesito siempre una API key? No, hay APIs públicas sin key (como la de GitHub para datos públicos). Pero la mayoría requiere autenticación.

? ¿Puedo modificar datos con GET? Técnicamente sí, pero **no debes**. GET es para leer. POST/PUT para modificar. Si usas GET para modificar, violas REST.

? ¿Qué es CORS y me afecta en Python? CORS es una restricción del navegador. En Python no te afecta (no hay navegador). Es cosa de JavaScript.

? ¿Qué significa que una API sea “sin estado” (stateless)? Que cada petición contiene toda la información necesaria. El servidor no guarda contexto entre peticiones. Como un camarero que no recuerda tu cara: cada vez le tienes que decir todo.

? ¿JSON o XML? Hoy en día, JSON gana por goleada. XML solo se usa en entornos legacy (bancos, SOAP).



Aprieta el lápiz

1. **GET a GitHub API:** Obtén los datos del usuario “python” y muestra su nombre real, repos públicos y seguidores.
2. **Parámetros de búsqueda:** Busca repositorios de Python con más de 1000 estrellas usando `/search/repositories`.
3. **POST a JSONPlaceholder:** Crea un post nuevo en `jsonplaceholder.typicode.com/posts` y muestra el ID devuelto.
4. **Manejo de errores:** Haz GET a una URL que no existe (404) y una que dé error 500. Muestra mensajes adecuados.

RAs cubiertos y criterios de evaluación

RA4 — Servicios en red (a-b)

Criterio	Descripción	Cubierto
RA4a	Utiliza APIs REST para obtener datos externos	✓
RA4b	Maneja peticiones HTTP y procesa respuestas JSON	✓

RA4c (servidores concurrentes) y RA4d (ThreadPool) se cubren en el **TEMA 10**. RA4e-g (asincio, disponibilidad, comparativa) se cubren en el **TEMA 11**.

TEMA 07 — APIs Comerciales

TEMA 07 — APIs Comerciales (RA4a-b)

“Las APIs de verdad no son de mentira. Te piden API key, te limitan las peticiones, se caen y tienes que manejar sus errores como un adulto.”

Índice

1. [API Key — el carnet de identidad](#)
 2. [Variables de entorno con dotenv](#)
 3. [OpenWeatherMap — el tiempo en tu ciudad](#)
 4. [Be the code, my friend, my friend — Consulta meteorológica](#)
 5. [OpenAI — el cerebro artificial](#)
 6. [Errores y rate limiting](#)
 7. [Reintentos inteligentes \(backoff\)](#)
 8.  [El ring de los conceptos — REST vs SOAP vs GraphQL](#)
 9. [Herramientas para probar APIs](#)
 10. [Preguntas tontas — APIs Comerciales](#)
 11.  [Aprieta el lápiz](#)
 12. [RAs cubiertos y criterios de evaluación](#)
-

API Key — el carnet de identidad

Las APIs comerciales te dan una **API key** (o token) que identifica quién eres y cuánto puedes usar.

```
import requests

API_KEY = "tu-api-key-aqui" # ⚠ NUNCA subas esto a GitHub

resp = requests.get(
    "https://api.openweathermap.org/data/2.5/weather",
    params={"q": "Sevilla", "appid": API_KEY, "units": "metric"}
)
```

Regla de oro: las API keys nunca van en el código. Usa variables de entorno.

Variables de entorno con dotenv

```
# pip install python-dotenv
from dotenv import load_dotenv
import os

load_dotenv() # Carga el archivo .env

API_KEY = os.getenv("OPENWEATHER_API_KEY")
if not API_KEY:
    raise ValueError("❌ Falta OPENWEATHER_API_KEY en .env")
```

.env (este archivo NO se sube a git):

```
OPENWEATHER_API_KEY=abc123...
OPENAI_API_KEY=sk-...
```

.gitignore:

```
.env
```

OpenWeatherMap — el tiempo en tu ciudad

```

import requests
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

def clima(ciudad):
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {"q": ciudad, "appid": API_KEY, "units": "metric", "lang": "es"}

    resp = requests.get(url, params=params)
    if resp.status_code != 200:
        print(f"❌ Error: {resp.json().get('message', 'desconocido')}")
        return

    datos = resp.json()
    print(f"☁️ {datos['name']}: {datos['main']['temp']}°C")
    print(f"    Sensación: {datos['main']['feels_like']}°C")
    print(f"    Humedad: {datos['main']['humidity']}%")
    print(f"    {datos['weather'][0]['description']}")

clima("Sevilla")

```

Respuesta JSON:

```

{
  "name": "Sevilla",
  "main": { "temp": 18.5, "feels_like": 17.2, "humidity": 65 },
  "weather": [{ "description": "cielo claro" }]
}

```

Be the code, my friend, my friend — Consulta meteorológica

“Sé el programa que pregunta por el tiempo, desde que escribes ‘python clima.py’ hasta que ves el resultado.”

```
$ python clima.py Barcelona
```

1. `load_dotenv()` → carga `.env` → obtiene `API_KEY`
2. Construye URL: `https://api.openweathermap.org/data/2.5/weather?q=Barcelona&appid=abc123...&units=metric&lang=es`
3. `requests.get()` → abre socket TCP
 - DNS lookup: `api.openweathermap.org` → `104.16.x.x`
 - Conexión TLS (HTTPS)
 - Envía petición HTTP GET
4. 🕒 Latencia de red (~200ms)
5. Recibe HTTP 200 + JSON:
HTTP/1.1 200 OK
Content-Type: application/json

```
{ "name": "Barcelona", "main": { "temp": 21.3 }, "weather": [ { "description": "nubes dispersas" } ] }
```
6. `resp.json()` → dict de Python
7. Extrae datos `['main']['temp'] = 21.3`
8. Extrae datos `['weather'][0]['description'] = "nubes dispersas"`
9. `print` → " 🌤️ Barcelona: 21.3°C"
10. `print` → " nubes dispersas" ❄️

OpenAI — el cerebro artificial

```

from openai import OpenAI
import os
from dotenv import load_dotenv

load_dotenv()

cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

respuesta = cliente.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "Eres un profesor de Python divertido."},
        {"role": "user", "content": "Explica qué es un Lock en 2 frases."}
    ],
    max_tokens=100,
    temperature=0.7
)

print(respuesta.choices[0].message.content)

```

Alternativa con httpx (sin librería oficial)

```

import httpx

API_KEY = os.getenv("OPENAI_API_KEY")
resp = httpx.post(
    "https://api.openai.com/v1/chat/completions",
    headers={"Authorization": f"Bearer {API_KEY}"},
    json={
        "model": "gpt-3.5-turbo",
        "messages": [{"role": "user", "content": "Hola!"}]
    }
)

print(resp.json()["choices"][0]["message"]["content"])

```

Errores y rate limiting

Las APIs comerciales **fallan**. Y fallan a menudo. Prepárate.


```

import requests

def llamada_segura(url, params, max_intentos=3):
    for intento in range(max_intentos):
        try:
            resp = requests.get(url, params=params, timeout=10)
            resp.raise_for_status() # Lanza excepción si 4xx o 5xx
            return resp.json()

        except requests.exceptions.Timeout:
            print(f"🕒 Timeout (intento {intento+1})")

        except requests.exceptions.HTTPError as e:
            codigo = e.response.status_code
            if codigo == 429: # Rate limit
                print("🌧 Demasiadas peticiones. Esperando...")
                import time
                time.sleep(5)
                continue
            elif codigo == 401:
                print("🔑 API key inválida")
                return None
            else:
                print(f"❌ HTTP {codigo}: {e.response.text}")
                return None

        except requests.exceptions.ConnectionError:
            print(f"🔌 Error de conexión (intento {intento+1})")
            import time
            time.sleep(2)

    return None

```

Reintentos inteligentes (backoff)

```
import time, requests

def conectar_con_backoff(url, params, max_intentos=5):
    for intento in range(max_intentos):
        try:
            resp = requests.get(url, params=params, timeout=5)
            resp.raise_for_status()
            return resp.json()
        except:
            espera = 2 ** intento # 1, 2, 4, 8, 16 segundos
            print(f"⚠️ Intento {intento+1} fallido. Esperando {espera}s...")
            time.sleep(espera)
    raise Exception("No se pudo conectar tras varios intentos")
```

El **backoff exponencial** evita saturar un servidor que ya está tocado.

El ring de los conceptos — REST vs SOAP vs GraphQL

REST: “Mira, soy el estándar. Simple, recursos, JSON. Todo el mundo me conoce.”

SOAP: “Yo fui el rey en los 2000. XML, WSDL, muy estructurado. Aún vivo en bancos.”

GraphQL: “Yo soy el moderno. Una sola URL, pides exactamente lo que necesitas, ni más ni menos.”

REST: “¿Y eso es bueno? Yo tengo URLs claras: /usuarios/5, /productos.”

GraphQL: “Sí, pero si quieres el nombre y el email del usuario y los títulos de sus posts, ¿cuántas peticiones necesitas?”

REST: “Dos: /usuarios/5 y /usuarios/5/posts.”

GraphQL: “Yo con una: query { usuario(id:5) { nombre email posts { titulo } } }.”

REST: “Vale, eres más eficiente. Pero yo soy más simple para empezar.”

GraphQL: “Tienes razón. Para proyectos pequeños, REST gana. Para grandes, yo escalo mejor.”

Herramientas para probar APIs

Herramienta	Tipo	Para qué
Postman	GUI	Probar APIs, colecciones, tests
curl	CLI	<code>curl https://api.github.com/users/python</code>
httpie	CLI moderno	<code>http https://api.github.com/users/python</code>
Insomnia	GUI	Alternativa open source a Postman

Preguntas tontas — APIs Comerciales

? **¿Me pueden banear por usar mucho la API?** Sí. Todas tienen límites (rate limits). Lee la documentación. OpenWeatherMap gratis: 60 peticiones/minuto.

? **¿Qué pasa si me paso del límite?** HTTP 429 (Too Many Requests). Algunas APIs bloquean tu IP por un tiempo.

? **¿OpenAI es caro?** GPT-3.5-turbo cuesta ~\$0.0015 por 1000 tokens. Una pregunta normal son ~100 tokens = \$0.00015. Muy barato para pruebas.

? **¿Puedo guardar la API key en el código para pruebas?** Solo si el código nunca va a GitHub. Mejor acostúmbrate a `.env` desde el principio.



Aprieta el lápiz

1. **Clima en 3 ciudades:** Usa OpenWeatherMap para mostrar el clima de Madrid, Barcelona y Sevilla a la vez.
2. **Chat con GPT:** Pregúntale a GPT-3.5 qué es un semáforo en Python y que te ponga un ejemplo.
3. **API con errores:** Conéctate a una URL que no existe y captura el error. Luego a una con API key falsa (401).
4. **Backoff test:** Crea una función que intente conectarse a un servidor que no existe y muestre el tiempo de espera entre intentos.

RAs cubiertos y criterios de evaluación

RA4 — Servicios en red (a-b, completo)

Criterio	Descripción	Cubierto
RA4a	Utiliza APIs REST para obtener datos externos	
RA4b	Maneja peticiones HTTP y procesa respuestas JSON	

RA4c (servidores concurrentes) y RA4d (ThreadPool) se cubren en el **TEMA 10**. RA4e-g (asincio, disponibilidad, comparativa) se cubren en el **TEMA 11**.

TEMA 08 — Hash y Cifrado Clásico

TEMA 08 — Hash y Cifrado Clásico (RA5)

“La criptografía es como las cerraduras: algunas son tan débiles que un soplo las abre (César), otras son tan seguras que ni un ejército las rompe (SHA-256).”

Índice

1. [Principios de seguridad](#)
2. [Hash — la huella digital](#)
3. [MD5, SHA-1, SHA-256 — ¿cuál usar?](#)
4. [Be the code, my friend, my friend — Hash de una contraseña](#)
5. [Hash con sal](#)
6.  [El ring de los conceptos — Hash vs Cifrado](#)
7. [Preguntas tontas — Hash](#)
8. [Cifrado César — el abuelo de la criptografía](#)
9. [Be the code, my friend, my friend — César paso a paso](#)
10.  [Aprieta el lápiz](#)
11. [RAs cubiertos y criterios de evaluación](#)

Principios de seguridad

Antes de cifrar nada, entiende estos principios. Son la base de todo.

Principio	Traducción al español de la calle
Zero Trust	No te fíes ni de tu sombra. Verifica siempre.
Mínimo privilegio	Da solo los permisos mínimos necesarios.
Defensa en profundidad	No pongas toda la seguridad en una capa.
Cifra todo	En tránsito (TLS) y en reposo (disco).
Rotación de claves	Cambia las claves periódicamente.

“La seguridad no es un producto, es un proceso.” — Bruce Schneier

Hash — la huella digital

Un **hash** es una función que convierte cualquier entrada en una cadena de longitud fija. Es **unidireccional**: no se puede revertir.

```
import hashlib

texto = b"Hola mundo"

print("MD5:", hashlib.md5(texto).hexdigest())
print("SHA1:", hashlib.sha1(texto).hexdigest())
print("SHA256:", hashlib.sha256(texto).hexdigest())
```

Salida:

```
MD5: 6cd3556deb0da54bca060b4c3947983f
SHA1: 182178c5daa62f6b5d17b1b08c6ff2c1f6959e29
SHA256: f2f09b3e8fbc885d8e2d5c5e2e8d7f9c6b7a5d4e3f2c1b0a9d8e7f6c5b4a3d2
```

Propiedades del hash

- **Determinista**: misma entrada → mismo hash
- **Unidireccional**: no se puede obtener el original
- **Longitud fija**: MD5=128 bits, SHA1=160 bits, SHA256=256 bits
- **Efecto avalancha**: un cambio mínimo cambia completamente el hash
- **Colisiones**: dos entradas diferentes con el mismo hash (casi imposible en SHA256)

MD5, SHA-1, SHA-256 — ¿cuál usar?

Algoritmo	Bits	¿Seguro?	Uso recomendado
MD5	128	✗ No	Solo checksums no críticos
SHA-1	160	✗ No	Solo legacy
SHA-256	256	✓ Sí	Todo uso general
SHA-512	512	✓ Sí	Cuando necesites más seguridad

```
import hashlib

# Hash de un archivo (verificar integridad)
with open("archivo.pdf", "rb") as f:
    hash_archivo = hashlib.sha256(f.read()).hexdigest()
    print(f"SHA256 del archivo: {hash_archivo}")
```

Nunca uses MD5 o SHA-1 para seguridad. Son vulnerables a colisiones. Usa SHA-256 o superior.

Be the code, my friend, my friend — Hash de una contraseña

“Sé el programa que registra a un usuario y luego verifica su login. Contraseñas seguras, sin almacenar la original.”

```

import hashlib

def registrar(usuario, contraseña):
    hash_contra = hashlib.sha256(contraseña.encode()).hexdigest()
    print(f" Usuario '{usuario}' registrado")
    print(f" Hash: {hash_contra[:16]}...")
    return hash_contra

def login(usuario, contraseña, hash_almacenado):
    hash_intento = hashlib.sha256(contraseña.encode()).hexdigest()
    if hash_intento == hash_almacenado:
        print(f" ✅ {usuario}: login correcto")
        return True
    else:
        print(f" ❌ {usuario}: contraseña incorrecta")
        return False

# Simular registro y login
hash_ana = registrar("Ana", "MiClaveSecreta123")
login("Ana", "MiClaveSecreta123", hash_ana) # ✅
login("Ana", "OtraClave", hash_ana) # ❌

```

Traza:

1. Ana se registra con "MiClaveSecreta123"
2. Python codifica a bytes → b"MiClaveSecreta123"
3. hashlib.sha256() procesa → 256 bits de hash
4. hexdigest() devuelve 64 caracteres hexadecimales
5. Guardamos en BD: "a1b2c3d4e5f6..."
6. Ana hace login con "MiClaveSecreta123"
7. Calculamos hash del intento → "a1b2c3d4e5f6..."
8. Comparamos con el almacenado → ¡COINCIDEN!
9. ✅ Login correcto
10. Un atacante intenta con "OtraClave"
11. Hash del intento → "ff34ee22..."
12. No coincide → ❌ Acceso denegado

La contraseña original **nunca** se almacena. Solo su hash.

Hash con sal

El ejemplo anterior tiene un problema: si Ana y Bob usan la misma contraseña, sus hashes son **idénticos**. Un atacante lo vería al instante y sabría que comparten clave.

La solución: añadir **sal** (salt) — un valor aleatorio distinto por usuario que se mezcla con la contraseña antes de hashear.

```
import hashlib, os

def registrar_con_salt(usuario, contraseña):
    salt = os.urandom(16) # 🧂 Salt: 16 bytes NUEVOS cada vez
    hash_contra = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    almacenado = salt.hex() + hash_contra # Guardamos: salt + hash juntos
    print(f" Usuario '{usuario}' registrado")
    print(f" Almacenado (salt+hash): {almacenado[:32]}...")
    return almacenado

def login_con_salt(usuario, contraseña, almacenado):
    # Del string guardado extraemos el salt (primera mitad) y el hash (resto)
    salt = bytes.fromhex(almacenado[:32]) # 🧂 RECUPERAMOS el mismo salt
    hash_original = almacenado[32:] # Hash que se guardó al registrar
    hash_intento = hashlib.sha256(salt + contraseña.encode()).hexdigest()
    if hash_intento == hash_original:
        print(f" ✅ {usuario}: login correcto")
        return True
    else:
        print(f" ❌ {usuario}: contraseña incorrecta")
        return False

# Simular — dos usuarios CON LA MISMA contraseña
hash_ana = registrar_con_salt("Ana", "clave123") # os.urandom genera salt_A
hash_bob = registrar_con_salt("Bob", "clave123") # os.urandom genera salt_B
            (distinto)

print(f"\n¿Son iguales los hashes? {hash_ana == hash_bob}") # ¡NO!
# salt_A + "clave123" ≠ salt_B + "clave123" → hashes distintos

login_con_salt("Ana", "clave123", hash_ana) # ✅ extrae salt_A, recalcula, coincide
login_con_salt("Ana", "otra", hash_ana) # ❌ extrae salt_A, recalcula, NO coincide
```

Salida:

```
Usuario 'Ana' registrado
Almacenado (salt+hash): a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6...
Usuario 'Bob' registrado
Almacenado (salt+hash): ffee00112233445566778899aabbccdde...
```

¿Son iguales los hashes? False

- ✅ Ana: login correcto
- ❌ Ana: contraseña incorrecta

La sal **se genera al registrar** y se guarda junto al hash. Al hacer login se recupera la misma sal del string almacenado. Así siempre podemos recalcular el hash exacto. Sin sal, dos usuarios con “clave123” tendrían el mismo hash y un atacante con tabla rainbow lo descubriría al instante.

Pool Puzzle — Verificación de contraseña con hash

¿Puedes ordenar estas líneas para crear un sistema de registro + login?

```
a) hash_login = hashlib.sha256((password + sal).encode()).hexdigest()
b) sal = os.urandom(16).hex()
c) return hash_guardado == hash_calculado
d) import hashlib, os
e) hash_guardado = hashlib.sha256((password + sal).encode()).hexdigest()
f) return hash_guardado + ":" + sal
g) def verificar(password, hash_guardado):
h) def registrar(password):
```

>  Solución

El ring de los conceptos — Hash vs Cifrado

Hash: — Yo soy la huella digital. Transformo cualquier texto en una cadena fija. No se puede deshacer. Unidireccional. Para siempre.

Cifrado: — Vaya, qué drástico. Yo puedo cifrar y descifrar. Tengo clave. Si tú pierdes un hash, no hay vuelta atrás. Yo puedo recuperar el mensaje original.

Hash: — ¡Esa es precisamente mi gracia! Para contraseñas no quieres que se pueda deshacer. Si alguien roba la base de datos, que se pegue con los hashes.

Cifrado: — Pero para enviar un mensaje secreto, el hash no sirve. Necesitas que el destinatario pueda leerlo. Ahí entro yo.

Hash: — Y para verificar integridad, nadie me gana. Un archivo, un hash. Si cambia un bit, el hash cambia por completo.

Cifrado: — Al final, cada uno a lo suyo. Tú para integridad y contraseñas; yo para confidencialidad.

Moraleja: El hash verifica integridad (no se puede deshacer). El cifrado protege confidencialidad (se puede deshacer con la clave). Ambos son necesarios.

Preguntas tontas — Hash

? ¿Se puede descifrar un hash? No, el hash no se descifra (es unidireccional). Pero se puede **adivinar** usando tablas rainbow o fuerza bruta. Por eso se usa **salt**: añadir un valor aleatorio antes de hashear.

? ¿Qué es una tabla rainbow? Una tabla precomputada de hashes para contraseñas comunes (123456, password, etc). Si tu hash está en la tabla, tu contraseña está comprometida. El **salt** lo evita porque añade aleatoriedad.

? ¿Qué pasa si dos usuarios tienen la misma contraseña? Sin salt, tendrían el mismo hash. Un atacante lo sabría. Con salt, aunque la contraseña sea la misma, los hashes son diferentes (mira el ejemplo de [Hash con sal](#) más arriba).

? ¿Para qué sirve el hash de un archivo? Para verificar integridad. Descargas Ubuntu, verificas su SHA256, y si coincide con el de la web oficial, sabes que no lo han manipulado.

Cifrado César — el abuelo de la criptografía

Julio César desplazaba cada letra 3 posiciones. El receptor, 3 hacia atrás.

```
def cifrar_cesar(texto, desplazamiento):
    resultado = ""
    for caracter in texto:
        if caracter.isalpha():
            base = ord('A') if caracter.isupper() else ord('a')
            resultado += chr((ord(caracter) - base + desplazamiento) % 26 + base)
        else:
            resultado += caracter
    return resultado

def descifrar_cesar(texto, desplazamiento):
    return cifrar_cesar(texto, -desplazamiento)

original = "Hola Mundo"
cifrado = cifrar_cesar(original, 3)
descifrado = descifrar_cesar(cifrado, 3)

print(f"Original: {original}")      # Hola Mundo
print(f"Cifrado: {cifrado}")        # Krod Pxqgr
print(f"Descifrado:{descifrado}")  # Hola Mundo
```

César NO es seguro. Solo 25 desplazamientos posibles. Se rompe en segundos. Pero es perfecto para aprender el concepto.

Be the code, my friend, my friend — César paso a paso

“Sé el cifrado César caracter a caracter.”

```
cifrar_cesar("Hola Mundo", 3)
```

1. Carácter 'H' (mayúscula)

```
→ base = ord('A') = 65  
→ (ord('H') - 65 + 3) % 26 + 65  
→ (72 - 65 + 3) % 26 + 65  
→ 10 % 26 + 65  
→ 10 + 65 = 75  
→ chr(75) = 'K'
```

2. Carácter 'o' (minúscula)

```
→ base = ord('a') = 97  
→ (111 - 97 + 3) % 26 + 97  
→ 17 % 26 + 97  
→ 17 + 97 = 114  
→ chr(114) = 'r'
```

3. 'l' → 'o'

4. 'a' → 'd'

5. ' ' → ' ' (no es letra, se queda igual)

6. 'M' → 'P'

7. 'u' → 'x'

8. 'n' → 'q'

9. 'd' → 'g'

10. 'o' → 'r'

Resultado: "Krod Pxqgr" 🎨



Aprieta el lápiz

1. **Compara hashes:** Calcula MD5, SHA1 y SHA256 de la palabra “python”. ¿Cuánto mide cada hash?
2. **Efecto avalancha:** Calcula SHA256 de “Hola mundo” y “Hola mundO” (cambia una ‘o’ por ‘O’). Compara.
3. **César personalizado:** Cifra “Hola mundo” con desplazamiento 5 y descifra el resultado.
4. **Fuerza bruta César:** Dado un texto cifrado con César (sin saber el desplazamiento), prueba los 25 desplazamientos hasta encontrar el que tenga sentido.
5. **Hash de archivo:** Calcula el SHA256 de un archivo .py de tu proyecto.

RAs cubiertos y criterios de evaluación

RA5 — Seguridad (parcial: hash, César, principios)

Criterio	Descripción	Cubierto
RA5a	Principios básicos de seguridad	✓
RA5c	Implementa funciones hash (MD5, SHA)	✓
RA5h	Conoce sistemas de roles y RBAC	✓ (principios)

RA5b (tipos de cifrado), RA5d (AES), RA5e (RSA), RA5f (firmas) y RA5g (cifrado híbrido) se cubren en el **TEMA 09**.

TEMA 09 — Cifrado Moderno

TEMA 09 — Cifrado Moderno (RA5)

“El cifrado moderno es como tener una caja fuerte con dos cerraduras: una rápida (AES) y otra segura para intercambiar llaves (RSA).”

Índice

1. [Cifrado simétrico — AES](#)
 2. [Componentes de AES](#)
 3. [Be the code, my friend, my friend — AES paso a paso](#)
 4. [Cifrado asimétrico — RSA](#)
 5.  [El ring de los conceptos — AES vs RSA](#)
 6. [Cifrado híbrido — lo mejor de ambos mundos](#)
 7. [Firmas digitales — demostrar autoría](#)
 8. [Roles y RBAC](#)
 9. [Be the code, my friend, my friend — Firma y verificación](#)
 10. [Preguntas tontas — Cifrado Moderno](#)
 11.  [Aprieta el lápiz](#)
 12. [RAs cubiertos y criterios de evaluación](#)
-

Cifrado simétrico — AES

Un solo cifrado con una sola clave. El que tiene la clave, puede descifrar.

```

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes

# Generar clave (32 bytes = 256 bits)
clave = get_random_bytes(32)

# Cifrar
cifrador = AES.new(clave, AES.MODE_EAX)
mensaje = b"Mensaje secreto"
texto_cifrado, tag = cifrador.encrypt_and_digest(mensaje)

print(f"Original: {mensaje}")
print(f"Cifrado (hex): {texto_cifrado.hex()}")
print(f"Nonce: {cifrador.nonce.hex()}")

# Descifrar
cifrador2 = AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce)
texto_descifrado = cifrador2.decrypt(texto_cifrado)
print(f"Descifrado: {texto_descifrado}")

```

AES es **simétrico**: misma clave para cifrar y descifrar. El problema es compartir esa clave de forma segura.

Componentes de AES

Componente	Descripción	¿Se envía?
Clave	16, 24 o 32 bytes (AES-128/192/256)	✗ Secreto
Nonce	Número aleatorio único	✓ Se envía con el cifrado
Tag	Código de autenticación (integridad)	✓ Se envía
Texto cifrado	El mensaje cifrado	✓ Se envía

Paquete enviado: [nonce 16B | tag 16B | cifrado ...]

Be the code, my friend, my friend — AES paso a paso

“Ana quiere enviar un mensaje a Bob. Traza cada byte.”

● ANA (emisora)

1. Tiene el mensaje: "Nos vemos a las 8"
2. Genera clave AES: `get_random_bytes(32)` → `b'\xa1\xb2\xc3...' (32 bytes)`
3. Crea cifrador AES modo EAX
 - `AES.new(clave, AES.MODE_EAX)`
 - Genera nonce automáticamente: `b'\x4f\x3a...' (16 bytes)`
4. Cifra:
 - `encrypt_and_digest(b"Nos vemos a las 8")`
 - Devuelve (texto_cifrado, tag)
 - `texto_cifrado = b'\x7f\xe2...' (ilegible)`
 - `tag = b'\x1a\xb2...' (16 bytes)`
5. Envía a Bob: nonce + tag + texto_cifrado

nonce (16B)	tag (16B)	cifrado
\x4f\x3a...	\x1a\xb2...	\x7f\xe2...

● BOB (receptor)

6. Recibe: nonce = primeros 16 bytes
tag = siguientes 16 bytes
cifrado = resto
7. `AES.new(clave, AES.MODE_EAX, nonce=recibido)`
8. `decrypt_and_verify(cifrado)`
 - Si el tag coincide → datos íntegros ✓
 - Si no → lanza excepción (alguien manipuló el mensaje)
9. Bob lee: "Nos vemos a las 8" ☒

Cifrado asimétrico — RSA

Dos claves: una **pública** (todos pueden verla) y una **privada** (solo tú).

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

# Generar par de claves
clave_rsa = RSA.generate(2048)
privada = clave_rsa.export_key()
publica = clave_rsa.publickey().export_key()

# Cifrar con clave PÚBLICA
cifrador = PKCS1_OAEP.new(RSA.import_key(publica))
mensaje = b"Mensaje secreto para Bob"
texto_cifrado = cifrador.encrypt(mensaje)

# Descifrar con clave PRIVADA
descifrador = PKCS1_OAEP.new(RSA.import_key(privada))
texto_descifrado = descifrador.decrypt(texto_cifrado)
print(f"Descifrado: {texto_descifrado}")
```

Característica	AES	RSA
Claves	Una	Dos (pública + privada)
Velocidad	Rápido (~1GB/s)	Lento (~1MB/s)
Tamaño máximo	Ilimitado	~190 bytes (con 2048 bits)
Distribución de clave	Problema	Fácil

RSA no sirve para cifrar mensajes grandes. Para eso necesitas **cifrado híbrido**.



El ring de los conceptos — AES vs RSA

AES: “Soy rapidísimo. Cifro un archivo entero en milisegundos. ¿El problema? Ambos necesitamos la misma clave.”

RSA: “Yo soy lento, pero elegante. Tú me das tu clave pública, yo cifro, y solo tú con tu privada puedes descifrar.”

AES: “Entonces, ¿para qué sirves si eres tan lento?”

RSA: “Para **distribuir tu clave**. Yo cifro tu clave AES con mi RSA. Tú la descifras con tu privada. Luego usamos AES para todo.”

AES: “O sea, ¿trabajamos en equipo?”

RSA: “Exacto. Eso se llama **cifrado híbrido**. Lo usan HTTPS, WhatsApp, Signal...”

Cifrado híbrido — lo mejor de ambos mundos



```

from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Random import get_random_bytes

# Claves de Bob
clave_bob = RSA.generate(2048)

# Ana cifra
clave_aes = get_random_bytes(32)
mensaje = b"Hola Bob, ¿quedamos mañana?"

cifrador_aes = AES.new(clave_aes, AES.MODE_EAX)
cifrado, tag = cifrador_aes.encrypt_and_digest(mensaje)

cifrador_rsa = PKCS1_OAEP.new(clave_bob.publickey())
clave_aes_cifrada = cifrador_rsa.encrypt(clave_aes)

# Se envía: (clave_aes_cifrada, nonce, tag, cifrado)

# Bob descifra
descifrador_rsa = PKCS1_OAEP.new(clave_bob)
clave_aes_recibida = descifrador_rsa.decrypt(clave_aes_cifrada)

descifrador_aes = AES.new(clave_aes_recibida, AES.MODE_EAX, nonce=cifrador_aes.nonce)
mensaje_descifrado = descifrador_aes.decrypt(cifrado)

print(f"Mensaje descifrado: {mensaje_descifrado.decode()}")

```

Este es el método que usa **HTTPS**: RSA para negociar la clave de sesión, AES para cifrar el tráfico.

Firmas digitales — demostrar autoría

Una **firma digital** demuestra que un mensaje fue creado por quien dice serlo y que no fue modificado.

Firmar (con clave privada):

```

from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA

clave = RSA.generate(2048)
mensaje = b"Este mensaje es de Ana"

h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave).sign(h)
print(f"Firma: {firma.hex()[:32]}...")

```

Verificar (con clave pública):

```

try:
    h = SHA256.new(mensaje)
    pkcs1_15.new(clave.publickey()).verify(h, firma)
    print("✅ Firma VÁLIDA – mensaje de Ana, no alterado")
except (ValueError, TypeError):
    print("❌ Firma INVÁLIDA – manipulada o no es de Ana")

```

Flujo completo:

1. Ana escribe mensaje
2. Calcula SHA256 del mensaje
3. Cifra el hash con su clave PRIVADA → esto es la firma
4. Envía: mensaje + firma
5. Bob recibe
6. Descifra firma con clave PÚBLICA de Ana → hash original
7. Calcula SHA256 del mensaje recibido
8. ¿Coinciden? → ✅ Es de Ana y nadie lo modificó

Roles y RBAC

RBAC = Role-Based Access Control. Los permisos dependen del **rol** del usuario.

```

class Usuario:
    def __init__(self, nombre, rol):
        self.nombre = nombre
        self.rol = rol

class Documento:
    def __init__(self, titulo, contenido):
        self.titulo = titulo
        self.contenido = contenido

PERMISOS = {
    "admin": ["leer", "escribir", "borrar", "compartir"],
    "editor": ["leer", "escribir"],
    "lector": ["leer"],
}

def puede(usuario, accion):
    return accion in PERMISOS.get(usuario.rol, [])

ana = Usuario("Ana", "admin")
bob = Usuario("Bob", "lector")

print(f"Ana puede borrar: {puede(ana, 'borrar')}")    # True
print(f"Bob puede borrar: {puede(bob, 'borrar')}")    # False

```

Rol	Leer	Escribir	Borrar	Compartir
admin	✓	✓	✓	✓
editor	✓	✓	✗	✗
lector	✓	✗	✗	✗

Be the code, my friend, my friend — Firma y verificación

“Sé el proceso de firma digital desde que Ana escribe hasta que Bob verifica.”

● ANA

1. Tiene mensaje: "Mañana a las 8 en el café"
2. SHA256 → 256 bits de hash
3. Cifra hash con su RSA privada → firma digital (256 bytes)
4. Envía a Bob: [mensaje] + [firma]

🚀 Por la red viaja: "Mañana a las 8 en el café" + 256 bytes de firma

● BOB

5. Recibe mensaje + firma
6. Calcula SHA256 del mensaje recibido
7. Descifra la firma con clave pública de Ana
8. Compara hashes...

¿Son iguales? → ☒ Mensaje de Ana
¿Son diferentes? → ☐ Algo va mal

9. Si alguien modificó el mensaje: hashes diferentes → ✗
10. Si no es de Ana: la clave pública no descifra la firma → ✗

Preguntas tontas — Cifrado Moderno

? ¿Qué clave uso para cada operación?

- Cifrar para alguien: usa su clave **pública**
- Descifrar: usa tu clave **privada**
- Firmar: usa tu clave **privada**
- Verificar firma: usa la clave **pública** del firmante

? ¿Qué es más seguro, AES-256 o RSA-2048? Ambos son seguros hoy en día. No compiten: son herramientas diferentes para problemas diferentes.

? ¿Qué pasa si pierdo mi clave privada? Pierdes acceso a todo lo cifrado con tu clave pública. Por eso se hacen **copias de seguridad** (y se guardan bien).

? ¿Puedo tener la misma clave RSA siempre? Sí, las claves no caducan. Pero por seguridad se recomienda rotarlas cada cierto tiempo (como cambiar la contraseña).

? ¿Cuánto tarda RSA en generar claves? Generar RSA 2048 bits lleva ~1-2 segundos. AES genera clave instantáneamente.



Aprieta el lápiz

1. **AES básico:** Cifra un mensaje con AES, luego descifralo. Muestra el nonce y el tag.
2. **RSA: cifra y descifra:** Genera un par RSA, cifra un mensaje corto y descifralo.
3. **Cifrado híbrido:** Cifra un mensaje largo con AES, cifra la clave AES con RSA. Simula el intercambio.
4. **Firma y verifica:** Firma un mensaje, luego modifícalo y comprueba que la verificación falla.
5. **RBAC:** Implementa un sistema con 3 roles (admin, editor, lector) y 4 acciones (leer, escribir, borrar, compartir).

RAs cubiertos y criterios de evaluación

RA5 — Seguridad (completo)

Criterio	Descripción	Cubierto
RA5a	Principios básicos de seguridad	✓
RA5b	Tipos de cifrado (simétrico, asimétrico)	✓
RA5c	Funciones hash	✓
RA5d	AES	✓
RA5e	RSA	✓
RA5f	Firmas digitales	✓
RA5g	Cifrado híbrido	✓
RA5h	Roles y RBAC	✓

TEMA 10 — Servidores Concurrentes

TEMA 10 — Servidores Concurrentes (RA4c-d)

“Un servidor secuencial atiende a un cliente cada vez. Los demás esperan. Un servidor concurrente atiende a todos a la vez. Como un camarero con 10 mesas.”

Índice

1. [El problema del servidor secuencial](#)
 2. [Be the code, my friend, my friend — Servidor secuencial vs concurrente](#)
 3. [Servidor con hilos \(threading\)](#)
 4. [ThreadPoolExecutor — control de recursos](#)
 5.  [Benchmark — Secuencial vs Hilos vs Pool](#)
 6. [Be the code, my friend, my friend — Servidor multihilo en acción](#)
 7.  [Pool Puzzle — Servidor concurrente](#)
 8.  [El ring de los conceptos — Hilo por cliente vs ThreadPool](#)
 9. [Preguntas tontas — Concurrencia](#)
 10.  [Aprieta el lápiz](#)
 11. [RAs cubiertos y criterios de evaluación](#)
-

El problema del servidor secuencial

```
import socket, time

def servidor_lento():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🌱 Servidor SECUENCIAL: 1 cliente cada vez")

    while True:
        conn, addr = srv.accept()
        print(f" Cliente {addr} conectado")
        time.sleep(3) # Simula trabajo pesado
        conn.sendall(b"Procesado\n")
        conn.close()
        print(f" Cliente {addr} atendido")
```

Si 3 clientes se conectan a la vez: el primero tarda 3s, el segundo 6s, el tercero 9s. 🌱

Be the code, my friend, my friend — Servidor secuencial vs concurrente

Servidor secuencial (3 clientes a la vez):

```
Tiempo: 0s — Cliente-1 conecta
        | Servidor procesa cliente-1 (3s)
3s — Cliente-1 listo
        | Cliente-2 conectó en t=0.1s, pero espera...
        | Servidor procesa cliente-2 (3s)
6s — Cliente-2 listo
        | Cliente-3 conectó en t=0.2s, pero espera...
        | Servidor procesa cliente-3 (3s)
9s — Cliente-3 listo ☒
```

Servidor concurrente (3 clientes a la vez):

```
Tiempo: 0s — Cliente-1 conecta → hilo-1 procesa
        |   Cliente-2 conecta → hilo-2 procesa
        |   Cliente-3 conecta → hilo-3 procesa
        |   (Los 3 procesan en paralelo)
3s — Cliente-1 listo 🏴
    |   Cliente-2 listo 🏴
    |   Cliente-3 listo 🏴
```

Con hilos, todos terminan a la vez. Sin hilos, el último espera 9s.

Servidor con hilos (threading)

Cada cliente recibe su propio hilo.

```
import socket, threading

def atender(conn, addr):
    print(f"[+] Cliente {addr} conectado")
    with conn:
        datos = conn.recv(1024)
        print(f"    Recibido: {datos.decode()}")
        import time
        time.sleep(2) # Simular trabajo
        conn.sendall(b"OK: " + datos)
    print(f"[-] Cliente {addr} desconectado")

def servidor_multihilo():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()
        print("🚀 Servidor MULTIHILLO – varios clientes a la vez")

        while True:
            conn, addr = srv.accept()
            hilo = threading.Thread(target=atender, args=(conn, addr))
            hilo.start()
```

ThreadPoolExecutor — control de recursos

Crear un hilo por cada cliente puede saturar el sistema con 10.000 conexiones. Un **ThreadPool** reutiliza un número fijo de hilos.

```
import socket, concurrent.futures

MAX_HILOS = 10

def atender(conn, addr):
    with conn:
        datos = conn.recv(1024)
        conn.sendall(b"OK: " + datos)

def servidor_pool():
    with socket.socket() as srv:
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        srv.bind(("127.0.0.1", 5000))
        srv.listen()

    with concurrent.futures.ThreadPoolExecutor(max_workers=MAX_HILOS) as pool:
        print(f" ⚡ Servidor con Pool de {MAX_HILOS} hilos")
        while True:
            conn, addr = srv.accept()
            pool.submit(atender, conn, addr)
```

Enfoque	Ventaja	Desventaja
Hilo por cliente	Simple	Puede saturar el SO (miles de hilos)
ThreadPool	Límite controlado	Si el pool se llena, los clientes esperan



Benchmark — Secuencial vs Hilos vs Pool

¿Cuánto más rápido es un servidor concurrente? Aquí tienes un experimento que lanza 10 clientes y mide el tiempo total:

```

import socket, time, threading, concurrent.futures

def cliente(id):
    """Conecta, envía 'ping', recibe 'pong'"""
    with socket.socket() as s:
        s.connect(("127.0.0.1", 5000))
        s.sendall(b"ping")
        s.recv(1024)

def prueba(tipo_servidor, n=10):
    """Lanza n clientes en paralelo y cronometra"""
    inicios = []
    with concurrent.futures.ThreadPoolExecutor(max_workers=n) as pool:
        futuros = [pool.submit(cliente, i) for i in range(n)]
        concurrent.futures.wait(futuros)

```

Enfoque	10 clientes (2s c/u)	Fórmula
 Secuencial	~20 segundos	$n \times \text{tiempo_por_cliente}$
 Hilos	~2 segundos	$\max(\text{tiempo_por_cliente})$
 ThreadPool (5 hilos)	~4 segundos	$\text{ceil}(n/\text{workers}) \times \text{tiempo_por_cliente}$

El ThreadPool con 5 hilos tarda el doble que hilos ilimitados, pero **no ahoga el sistema**. Con 1000 clientes, hilos ilimitados matarían el PC; el pool encola y sirve de a 5.

Cliente de prueba — lanzador masivo

```

import socket, threading, time

def cliente(id):
    try:
        with socket.socket() as s:
            s.connect(("127.0.0.1", 5000))
            s.sendall(f"Cliente-{id}\n".encode())
            resp = s.recv(1024)
            print(f"    ✅ Cliente-{id}: {resp.decode().strip()}")
    except Exception as e:
        print(f"    ❌ Cliente-{id}: {e}")

print("Lanzando 10 clientes...")
hilos = []
for i in range(10):
    t = threading.Thread(target=cliente, args=(i,))
    t.start()
    hilos.append(t)
for t in hilos:
    t.join()
print("🏁 Todos los clientes terminaron")

```

Este script prueba que tu servidor concurrente realmente atiende a varios clientes a la vez. Si es secuencial, los clientes 2-9 esperarán su turno.

“Sé el servidor que recibe 3 clientes a la vez. Traza cada hilo.”

● SERVIDOR PRINCIPAL

1. Crea socket, bind(5000), listen()
2. accept() → espera... ⌚

[Cliente-1: t=0s]

3. accept() → conn1, addr1
4. Crea hilo-1 → start(atender, conn1)
5. Vuelve a accept() inmediatamente

[Cliente-2: t=0.1s]

6. accept() → conn2, addr2
7. Crea hilo-2 → start(atender, conn2)
8. Vuelve a accept()

[Cliente-3: t=0.2s]

9. accept() → conn3, addr3
10. Crea hilo-3 → start(atender, conn3)
11. Vuelve a accept()

AHORA 4 HILOS EJECUTANDO:

Hilo ppal	accept() esperando más clientes
Hilo-1	recibe → procesa (2s) → responde
Hilo-2	recibe → procesa (2s) → responde
Hilo-3	recibe → procesa (2s) → responde

Los 3 clientes son atendidos en paralelo.

A los ~2s, todos reciben respuesta. 🍷

🧩 Pool Puzzle — Servidor con ThreadPool

Desordena y reconstruye este servidor que usa ThreadPoolExecutor:

```
a) from concurrent.futures import ThreadPoolExecutor
b)     pool.submit(atender, conn, addr)
c)     with ThreadPoolExecutor(max_workers=5) as pool:
d) import socket
e) def atender(conn, addr):
f)     conn.sendall(b"OK\n")
g) while True:
h)     with socket.socket() as srv:
i)         datos = conn.recv(1024)
j)         conn, addr = srv.accept()
k)         srv.bind(("0.0.0.0", 5000))
l)         srv.listen()
```

>  Solución

El ring de los conceptos — Hilo por cliente vs ThreadPool

HiloPorCliente: — Soy el enfoque clásico. Llego un cliente, creo un hilo, lo atiendo, el hilo muere. Sencillo y directo.

ThreadPool: — ¿Y si llegan 1000 clientes? Creas 1000 hilos y el sistema se colapsa. Yo tengo un número fijo de hilos, como un equipo de camareros limitado.

HiloPorCliente: — Pero cada cliente tiene su hilo dedicado. No esperan. Es más justo.

ThreadPool: — Justo pero ineficiente. Con mi pool, los clientes esperan un poco pero el sistema no se ahoga. Además, reutilizo hilos, evitando el coste de crearlos y destruirlos constantemente.

HiloPorCliente: — Para pocos clientes simultáneos, soy más simple de implementar.

ThreadPool: — Y yo escalo mejor. Para servidores en producción, soy la opción sensata.

Moraleja: Hilo por cliente es simple y funciona para pocos clientes. ThreadPool escala mejor y controla los recursos. En producción, usa ThreadPool.

Preguntas tontas — Concurrencia

? **¿Cuántos hilos puede tener un servidor?** Depende del SO. En Windows, unos pocos miles. Pero más allá de ~100, el cambio de contexto (context switch) perjudica el rendimiento.

? **¿Qué pasa si un hilo se cuelga?** Ese hilo queda bloqueado, pero los demás siguen. El problema es si no libera el socket (fuga de recursos).

? **¿ThreadPool o hilo por cliente en producción?** ThreadPool siempre. Controlas cuántos hilos se crean. Hilo por cliente solo para prototipos.

? **¿Es seguro compartir variables globales entre hilos del servidor?** No sin Lock. Si dos hilos modifican la misma variable, usa Lock. Mejor aún: evita compartir estado.

? **¿Puedo mezclar hilos y asyncio?** Sí, con `loop.run_in_executor()`. Pero empieza con uno u otro.

? **¿Y si el servidor recibe 10.000 conexiones?** ThreadPool con 100 hilos + cola de espera. O usa **asyncio** (TEMA 11) que escala mejor.



Aprieta el lápiz

1. **Servidor secuencial → hilos:** Convierte un servidor TCP secuencial en uno multihilo.
2. **ThreadPool:** Implementa el mismo servidor con ThreadPoolExecutor de 5 hilos.
3. **Lanzador de clientes:** Crea un script que lance 10 clientes simultáneos para probar tu servidor.
4. **Contador de conexiones:** El servidor debe mostrar cuántos clientes han sido atendidos (usa Lock para la variable compartida).

RAs cubiertos y criterios de evaluación

RA4 — Servicios en red (c-d)

Criterio	Descripción	Cubierto
RA4c	Implementa servidores concurrentes con hilos	✓
RA4d	Gestiona pools de hilos (ThreadPoolExecutor)	✓

RA4a-b (APIs REST y comerciales) se cubren en los **TEMAS 06-07**. RA4e-g (asyncio, disponibilidad, comparativa) se cubren en el **TEMA 11**.

TEMA 11 — Asyncio y Disponibilidad

TEMA 11 — Asyncio y Disponibilidad (RA4e-g)

“Asyncio es como un cocinero que, mientras espera a que hierva el agua, corta verduras. En vez de quedarse mirando la olla, hace otras cosas.”

Índice

1. [El problema de esperar](#)
2. [Asyncio — fundamentos](#)
3. [Corrutinas y event loop](#)
4. [Servidor con asyncio](#)
5. [Be the code, my friend, my friend — Asyncio paso a paso](#)
6. [Disponibilidad — heartbeat](#)
7. [Reintentos con backoff](#)
8. [Timeout en servidor](#)
9.  [El ring de los conceptos — Threads vs Asyncio](#)
10. [Preguntas tontas — Asyncio](#)
11.  [Aprieta el lápiz](#)
12. [RAs cubiertos y criterios de evaluación](#)

El problema de esperar

Un servidor que usa `accept()` y `recv()` se **bloquea** mientras espera. No puede hacer nada más.

```
# ❌ Esto bloquea TODO el programa
datos = conn.recv(1024) # El programa se para aquí hasta que lleguen datos
```

Solución 1: Hilos (TEMA 10) — caros si hay muchos clientes.

Solución 2: Asyncio — un solo hilo, pero cambia de tarea cuando una espera.

Asyncio — fundamentos

```
import asyncio

async def saludar():
    print("Hola")
    await asyncio.sleep(1) # "Oye, mientras duermo, haz otras cosas"
    print("Mundo")

asyncio.run(saludar())
```

Concepto	Qué es
<code>async def</code>	Define una corrutina (función que puede pausarse)
<code>await</code>	Pausa la corrutina hasta que algo termine
<code>asyncio.run()</code>	Crea el event loop y ejecuta la corrutina principal

`await` = "Oye, esto va a tardar. Mientras, ocúpate de otras cosas."

Corrutinas y event loop

El **event loop** es el gestor. Decide qué corrutina ejecuta en cada momento.

```
import asyncio

async def tarea(nombre, segundos):
    print(f" {nombre} empieza")
    await asyncio.sleep(segundos)
    print(f" {nombre} termina ({segundos}s)")

async def main():
    # Lanzar 3 tareas "a la vez"
    await asyncio.gather(
        tarea("A", 3),
        tarea("B", 1),
        tarea("C", 2)
    )

asyncio.run(main())
```

Salida:

```
A empieza
B empieza
C empieza
B termina (1s)
C termina (2s)
A termina (3s)
```

Las 3 empiezan a la vez. B termina primero (solo 1s). El event loop aprovecha los await de otras para avanzar.

Servidor con asyncio

```

import asyncio

async def atender(reader, writer):
    addr = writer.get_extra_info('peername')
    print(f"[+] Cliente {addr} conectado")

    datos = await reader.read(1024)
    print(f"    Recibido: {datos.decode()}")

    writer.write(b"OK: " + datos)
    await writer.drain() # Espera a que se envíe

    writer.close()
    await writer.wait_closed()
    print(f"[-] Cliente {addr} desconectado")

async def main():
    servidor = await asyncio.start_server(atender, "127.0.0.1", 5000)
    print("🚀 Servidor ASYNCIO en 127.0.0.1:5000")

    async with servidor:
        await servidor.serve_forever()

asyncio.run(main())

```

Con asyncio, un solo hilo puede atender miles de conexiones. Cada await es una oportunidad para atender a otro cliente.

Be the code, my friend, my friend — Asyncio paso a paso

“Sé el event loop. Tu trabajo es coordinar corrutinas sin bloquear ni un milisegundo.”

Event Loop arranca

```
|
|— 1. Ejecuta main()
|   |— Crea servidor TCP
|   |— Registra atender() para nuevos clientes
|
|— 2. Event Loop: "Espero eventos... (I/O, timers, etc.)"
|
|— [Cliente-1 conecta]
| 3. Event Loop: "¡Cliente nuevo! Ejecuto atender(cliente1)"
| 4. atender(cliente1) empieza
| 5. await reader.read() → "No hay datos aún"
| 6. atender(cliente1) se pausa (cede el control)
|
|— [Cliente-2 conecta mientras cliente1 espera]
| 7. Event Loop: "¡Otro cliente! Ejecuto atender(cliente2)"
| 8. atender(cliente2) empieza
| 9. await reader.read() → "Tampoco hay datos"
| 10. atender(cliente2) se pausa
|
|— [Cliente-1 envía datos]
| 11. Event Loop: "Cliente1 tiene datos → reanudo atender(cliente1)"
| 12. atender(cliente1) recibe los datos
| 13. writer.write() → escribe buffer
| 14. await writer.drain() → espera envío → se pausa
|
|— [Cliente-2 envía datos]
| 15. Event Loop: "Cliente2 tiene datos → reanudo atender(cliente2)"
| 16. atender(cliente2) recibe, responde, termina ☒
|
|— [writer.drain() de cliente1 listo]
| 17. Event Loop: "Cliente1 puede finalizar"
| 18. atender(cliente1) termina ☒
|
|— Event Loop sigue esperando más clientes...
```

Nunca hay espera activa. Cuando una corrutina espera, otra aprovecha. **Un solo hilo, miles de conexiones.**

Disponibilidad — heartbeat

Un **heartbeat** (latido) es un mensaje periódico para verificar que el servidor sigue vivo.

```
import asyncio

async def heartbeat(intervalo=5):
    while True:
        print("💖 Heartbeat: servidor vivo")
        await asyncio.sleep(intervalo)

async def servidor_con_heartbeat():
    # Lanzar heartbeat en segundo plano
    asyncio.create_task(heartbeat())

    servidor = await asyncio.start_server(
        lambda r, w: None, "127.0.0.1", 5000
    )
    async with servidor:
        await servidor.serve_forever()

asyncio.run(servidor_con_heartbeat())
```

Reintentos con backoff


```

import asyncio

async def conectar_con_backoff(host, port, max_intentos=5):
    for intento in range(max_intentos):
        try:
            reader, writer = await asyncio.wait_for(
                asyncio.open_connection(host, port),
                timeout=3
            )
            print(f"✅ Conectado a {host}:{port}")
            return reader, writer
        except (asyncio.TimeoutError, ConnectionRefusedError):
            espera = 2 ** intento # 1, 2, 4, 8, 16 segundos
            print(f"⚠️ Intento {intento+1} fallido. Esperando {espera}s...")
            await asyncio.sleep(espera)

    raise Exception("No se pudo conectar")

```

El **backoff exponencial** evita saturar un servidor que ya está teniendo problemas.

Timeout en servidor

```

import asyncio

async def atender_con_timeout(reader, writer):
    try:
        # Esperar datos con timeout de 10 segundos
        datos = await asyncio.wait_for(reader.read(1024), timeout=10)
        writer.write(b"OK: " + datos)
        await writer.drain()
    except asyncio.TimeoutError:
        writer.write(b"🕒 Timeout: conexión cerrada por inactividad\n")
        await writer.drain()
    finally:
        writer.close()
        await writer.wait_closed()

```



El ring de los conceptos — Threads vs Asyncio

Thread: “Yo soy multitarea de verdad. Tengo mi propia pila, mi propio contexto. El SO me gestiona.”

Asyncio: “Yo soy multitarea cooperativa. Un solo hilo, pero cambio de tarea cuando una espera.”

Thread: “Si tengo 10.000 clientes, creo 10.000 hilos. El sistema sufre.”

Asyncio: “Yo con 10.000 clientes uso un hilo y 10.000 corrutinas. Mucho más ligero.”

Thread: “Pero mis operaciones son bloqueantes de verdad. Si llamo a `time.sleep()`, otro hilo ejecuta.”

Asyncio: “Mis operaciones son `await` — nunca bloqueo. El event loop decide qué toca.”

Thread: “Para servidores pequeños (<100 clientes), soy más simple.”

Asyncio: “Para servidores con mucho I/O y muchas conexiones, soy imbatible.”

Característica	Threads	Asyncio
No de hilos	Varios (gestión del SO)	1 (event loop)
Cambio de contexto	Gestionado por el SO	Cooperativo (en <code>await</code>)
Escalabilidad	Media (límite de hilos)	Alta (miles de conexiones)
Complejidad	Baja (simple)	Media (curva de aprendizaje)
CPU-bound	No sirve (GIL)	No sirve
I/O-bound	Sí funciona	Excelente

Preguntas tontas — Asyncio

? ¿Asyncio es más rápido que threads? Para I/O-bound tasks, sí, porque no hay cambio de contexto del SO. Para CPU-bound, no hay diferencia (ambos limitados por el GIL).

? ¿Puedo mezclar código síncrono con asyncio? Sí, con `loop.run_in_executor()`. Pero mejor si todo es asyncio.

? **¿Qué es una corrutina?** Una función declarada con `async def` que puede pausarse con `await` y reanudarse después. No es un hilo, es una función que sabe esperar.

? **¿await bloquea el hilo?** No. `await` le dice al event loop: “ahora no necesito CPU, ocúpate de otras corrutinas”. Es **cooperativo**.

? **¿Cuándo usar threads y cuándo asyncio?**

- Threads: proyectos pequeños, librerías bloqueantes, simplicidad
- Asyncio: muchos clientes concurrentes, mucho I/O, escalabilidad

Aprieta el lápiz

1. **Asyncio básico:** Crea 3 corrutinas que esperen 1, 2 y 3 segundos. Lánzalas con `gather` y mide el tiempo total.
2. **Servidor asyncio:** Convierte el servidor TCP del TEMA 10 a asyncio.
3. **Heartbeat:** Añade un heartbeat que imprima “ vivo” cada 3s mientras el servidor funciona.
4. **Backoff:** Crea un cliente que intente conectarse 3 veces con backoff exponencial.
5. **Comparativa:** Mide el tiempo de atender 100 clientes con threads vs asyncio (simula 0.1s de I/O).

RAs cubiertos y criterios de evaluación

RA4 — Servicios en red (e-g)

Criterio	Descripción	Cubierto
RA4e	Implementa mecanismos de disponibilidad (heartbeat, reintentos, timeout)	
RA4f	Desarrolla servidores con asyncio	
RA4g	Compara modelos de concurrencia (hilos vs asyncio)	

RA4c (servidores concurrentes con hilos) y RA4d (ThreadPool) se cubren en el **TEMA 10**.



INICIAL RESUELTO 1 — Procesos y Subprocess

1. Mi primer PID

```
import os
print(f"Mi PID es {os.getpid()}")
```

`os.getpid()` devuelve el identificador único del proceso actual.

2. Versión de Python

```
import subprocess
resultado = subprocess.run(["python", "--version"], capture_output=True, text=True)
print(resultado.stdout.strip())
```

`capture_output=True` captura la salida. `text=True` la devuelve como string.

3. Abrir bloc de notas

```
import subprocess, time
notepad = subprocess.Popen(["notepad.exe"])
print(f"PID: {notepad.pid}")
time.sleep(3)
notepad.terminate()
```

`Popen` no espera. `terminate()` envía señal de cierre.



INICIAL POR RESOLVER 1 — Procesos y Subprocess

1. Listar el directorio actual

Crea un programa que use `subprocess.run` para ejecutar el comando `dir` (Windows) o `ls` (Linux) y muestre la salida completa por pantalla.

2. Saber el nombre del equipo

Usa `subprocess.run` con el comando `hostname` para capturar y mostrar el nombre de tu máquina.

3. Abrir la calculadora

Usa `subprocess.Popen` para abrir la calculadora (`calc.exe`). El programa debe mostrar el PID, esperar 2 segundos y cerrarla con `terminate()`.



INTERMEDIO RESUELTO 1 — Procesos y Subprocess

4. Comando con error

```
import subprocess
resultado = subprocess.run(["python", "--versio"], capture_output=True, text=True)
print(f"Código: {resultado.returncode}")
print(f"Error: {resultado.stderr}")
```

El código de retorno != 0 indica error.

5. Ping con timeout

```
import subprocess
try:
    resultado = subprocess.run(["ping", "google.com", "-n", "3"],
                               capture_output=True, text=True, timeout=5)
    print(resultado.stdout)
except subprocess.TimeoutExpired:
    print("El ping tardó demasiado")
```

timeout lanza TimeoutExpired si excede.

6. Crear carpeta

```
import subprocess
# En Windows: cmd /c ejecuta comandos internos del shell
resultado = subprocess.run(["cmd", "/c", "mkdir", "prueba_psp"],
                           capture_output=True, text=True)
print(f"Creada con código: {resultado.returncode}")
```

En Linux usarías directamente ["mkdir", "prueba_psp"] porque mkdir es un ejecutable real.



INTERMEDIO POR RESOLVER 1 —

Procesos y Subprocess

4. Filtrar ipconfig

Ejecuta `ipconfig` con `subprocess.run`, captura toda la salida y muestra solo las líneas que contengan “IPv4”.

5. Comprobar si un programa existe

Usa `subprocess.run` con el comando `where python` (Windows) o `which python` (Linux) para comprobar si Python está accesible desde la terminal. Muestra el código de retorno.

6. Abrir el navegador

Usa `subprocess.Popen` para abrir el navegador predeterminado con la URL `http://localhost:4321`. En Windows usa `start http://localhost:4321` con `shell=True`, en Linux usa `xdg-open http://localhost:4321`.

★ AVANZADO 1 — Procesos y Subprocess

★ AVANZADO 01 — Procesos y Subprocess

1. 🎯 Lanzador múltiple

Crea un programa que lance 3 programas distintos a la vez (notepad, calc, mspaint). Muestra sus PIDs y luego los mata todos a los 5 segundos.

Pista: Guarda los objetos Popen en una lista o tupla junto con el nombre de cada programa. Recorre la lista para mostrar PIDs y más tarde para terminarlos con `terminate()`.

2. 🔍 Captura de ping

Ejecuta `ping 8.8.8.8 -n 5`, captura stdout y extrae solo las líneas que contienen “tiempo” o “time”.

Pista: Divide el stdout con `split("\n")` y filtra cada línea con el operador `in` para buscar las palabras clave.

3. 🧩 PID del padre

Crea un programa que muestre su propio PID y el PID del proceso padre (PPID).

Pista: Usa `os.getpid()` para tu PID y `os.getppid()` para el PPID. Ejecuta desde la terminal y observa quién es el padre.

4. 🐱🐶 Comunicación en dos direcciones

Crea un proceso hijo que reciba texto por stdin y devuelva el texto en mayúsculas.

Pista: Usa `subprocess.Popen` con `stdin=subprocess.PIPE`, `stdout=subprocess.PIPE` y `text=True`. Luego llama a `communicate(input="tu texto")` para enviar datos y leer la respuesta.

5. 🕒 Timeout con reintentos

Crea una función que ejecute un comando con timeout de 3s. Si falla, reintenta hasta 3 veces. Si al final no funciona, lanza una excepción.

Pista: Envuelve `subprocess.run` con `timeout=3` y `check=False` dentro de un bucle `for`. Captura `subprocess.TimeoutExpired` en cada intento y rompe el bucle si el comando se ejecuta correctamente. Si se agotan los intentos, lanza una excepción personalizada.

6. 🕒 Mini monitor de procesos

Crea un programa que lance un `subprocess` y verifique su estado cada segundo mientras está vivo.

Pista: `proc.poll()` devuelve `None` si el proceso sigue ejecutándose, o el código de retorno si ya terminó. Úsalo en un bucle `while proc.poll() is None` con `time.sleep(1)` dentro.



INICIAL RESUELTO 2 — Hilos

Fundamentos

1. Hilo que saluda

```
import threading

def saludar():
    print("Hola desde un hilo")

h = threading.Thread(target=saludar)
h.start()
h.join()
```

Nunca olvides `start()` para lanzar y `join()` para esperar.

2. Dos hilos

```
import threading

def hilo_a():
    for _ in range(3):
        print("Hilo A")

def hilo_b():
    for _ in range(3):
        print("Hilo B")

h1 = threading.Thread(target=hilo_a)
h2 = threading.Thread(target=hilo_b)
h1.start()
h2.start()
h1.join()
h2.join()
```

Los prints se entremezclarán. El orden no está garantizado.

3. Hilo con nombre

```
import threading
def mostrar():
    print(f"Soy {threading.current_thread().name}")
h = threading.Thread(target=mostrar, name="MiHilo")
h.start()
h.join()
```

`current_thread().name` funciona desde dentro del hilo.

INICIAL POR RESOLVER 2 — Hilos

Fundamentos

1. Hilo que cuenta

Crea un hilo que ejecute una función que cuente del 1 al 5 imprimiendo cada número. Lánzalo con `start()` y espera con `join()`.

2. Múltiples hilos con retardo

Crea 3 hilos que impriman su número (1, 2, 3) y luego duerman 1 segundo con `time.sleep(1)`. Lánzalos todos y espera a que terminen.

3. Hilo con argumento

Crea una función que reciba un nombre como argumento e imprima “Hola, {nombre}”. Lanza un hilo pasándole el argumento con `args=("Ana",)`.



INTERMEDIO RESUELTO 2 — Hilos

Fundamentos

4. join() o no join()

```
import threading, time
def lento():
    for i in range(5):
        print(i)
        time.sleep(1)
h = threading.Thread(target=lento)
h.start()
# Sin join() → el programa principal termina antes
print("Fin del programa principal")
```

Sin join(), el programa principal no espera.

5. Hilo daemon

```
import threading, time
def tic():
    while True:
        print("tic")
        time.sleep(1)
h = threading.Thread(target=tic, daemon=True)
h.start()
time.sleep(3)
print("Programa termina — el daemon muere")
```

El daemon se mata al salir del programa principal.

6. Timer

```
import threading
def despierta():
    print("¡Despierta!")
t = threading.Timer(4.0, despierta)
t.start()
```

Timer ejecuta UNA SOLA vez después del retardo.



INTERMEDIO POR RESOLVER 2 — Hilos

Fundamentos

4. Join con timeout

Crea un hilo que duerma 5 segundos. En el hilo principal, usa `join(timeout=2)` y comprueba con `is_alive()` si el hilo sigue ejecutándose después del timeout.

5. Listar hilos activos

Crea 3 hilos que hagan tareas distintas. Usa `threading.enumerate()` para listar todos los hilos activos y muestra sus nombres.

6. Identificar el hilo actual

Crea una función que muestre `threading.current_thread().name`, `threading.current_thread().ident` y `threading.current_thread().daemon`. Ejecútala desde el hilo principal y desde un hilo secundario.

★ AVANZADO 2 — Hilos Fundamentos

★ AVANZADO 02 — Hilos Fundamentos

1. 🎯 Carrera de mensajes

Crea 5 hilos, cada uno imprime su nombre 10 veces. Observa el orden impredecible.

Pista: Usa una lista por comprensión para crear los 5 hilos. Lanza todos con un bucle `for h in hilos: h.start()` y espera con `for h in hilos: h.join()`. El orden de salida cambiará en cada ejecución.

2. 🔍 Simular descarga con progreso

Crea un hilo que descargue (simulado con `sleep`) y muestre progreso cada 25%.

Pista: Dentro de la función, itera con `range(0, 101, 25)`, usa `time.sleep(0.5)` para simular el tiempo de descarga e imprime el porcentaje en cada paso.

3. 🚧 Detener hilo con bandera

Crea un hilo infinito que se detenga cuando pulses Enter.

Pista: Declara una variable booleana global `ejecutando = True`. El hilo comprueba la variable en un `while ejecutando:`. Desde el principal, usa `input()` para esperar Enter y luego cambia `ejecutando = False`.

4. 🤖 Hilo que devuelve un valor

Usa una variable compartida para que un hilo devuelva un resultado al principal.

Pista: Los hilos no tienen `return`. Usa una lista compartida como contenedor: el hilo guarda el resultado con `lista.append(valor)` y el principal lo lee tras `hilo.join()`.

5. ⌚ Timer con repetición

El Timer normal solo se dispara una vez. Crea un timer que se repita cada 2 segundos usando recursividad.

Pista: Dentro de la función que ejecuta el Timer, crea otro `threading.Timer(2.0, funcion).start()` al final. Así la función se programa a sí misma de nuevo.

6. 🕒 Pool de hilos manual

Crea un “pool” manual con una cola de tareas. 2 hilos cogen tareas de la cola y las ejecutan.

Pista: Usa `queue.Queue` (thread-safe) para las tareas. Los trabajadores ejecutan un bucle llamando a `cola.get()`. Al terminar cada tarea llaman a `cola.task_done()`. El principal espera con `cola.join()`. Crea los workers como `daemon=True` para que no bloqueen la salida, o usa un valor centinela (`None`) en la cola para que terminen ordenadamente.



INICIAL RESUELTO 3 — Sincronización entre Hilos

1. Carrera sin Lock

```
import threading
contador = 0
def inc():
    global contador
    for _ in range(1000):
        contador += 1
hilos = [threading.Thread(target=inc) for _ in range(2)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
print(contador) # ❌ No será 2000
```

Sin Lock, hay condición de carrera. El valor será < 2000 .

2. Carrera con Lock

```
import threading
contador = 0
lock = threading.Lock()
def inc():
    global contador
    for _ in range(1000):
        with lock:
            contador += 1
hilos = [threading.Thread(target=inc) for _ in range(2)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
print(contador) # ✅ 2000
```

with lock: garantiza exclusión mutua.

3. Semáforo simple

```
import threading, time
s = threading.Semaphore(2)
def entrar(id):
    print(f"{id} espera")
    with s:
        print(f" → {id} dentro")
        time.sleep(1)
    print(f" ← {id} sale")
hilos = [threading.Thread(target=entrar, args=(i,)) for i in range(4)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Solo 2 hilos entran a la vez. Los demás esperan.



INICIAL POR RESOLVER 3 —

Sincronización entre Hilos

1. Lock protege una lista

Crea 2 hilos que añadan elementos a una lista compartida. Uno añade “A” 5 veces, el otro “B” 5 veces. Usa un Lock para evitar mezclas.

2. RLock básico

Crea una función que adquiera un RLock dos veces desde el mismo hilo (llamando a otra función que también lo adquiera). Comprueba que no se produce deadlock.

3. Semáforo con timeout

Crea un Semaphore(2) y lanza 4 hilos que intenten entrar. Cada hilo espera como máximo 1 segundo con `acquire(timeout=1)`. Los que no consigan entrar deben mostrar “timeout”.



INTERMEDIO RESUELTO 3 —

Sincronización entre Hilos

4. Barrera de 3

```
import threading, time
b = threading.Barrier(3)
def corredor(id):
    time.sleep(id)
    print(f" {id} preparado")
    b.wait()
    print(f"🏃 {id} salió")
hilos = [threading.Thread(target=corredor, args=(i,)) for i in range(3)]
for h in hilos:
    h.start()
for h in hilos:
    h.join()
```

Todos imprimen “preparado” antes de que nadie “salga”.

5. Productor-Consumidor básico

```
import threading, time, random
cola = []
c = threading.Condition()
def productor():
    for _ in range(3):
        with c:
            cola.append(random.randint(1,10))
            print(f"Produjo {cola[-1]}")
            c.notify()
            time.sleep(1)
def consumidor():
    for _ in range(3):
        with c:
            while not cola: c.wait()
            print(f"Consumió {cola.pop(0)}")
threading.Thread(target=productor).start()
threading.Thread(target=consumidor).start()
```

`wait()` espera hasta que `notify()` lo despierte.

6. RLock o Lock

```
import threading
lock = threading.Lock()
lock.acquire()
# lock.acquire() # ❌ DEADLOCK – el hilo se espera a sí mismo

rlock = threading.RLock()
rlock.acquire() # ✅ ok
rlock.acquire() # ✅ ok (mismo hilo)
rlock.release()
rlock.release()
```

RLock permite al mismo hilo adquirirlo varias veces.



INTERMEDIO POR RESOLVER 3 —

Sincronización entre Hilos

4. Condition con notify_all

Crea 3 hilos consumidores que esperen en una `Condition`. Un hilo productor añade 3 elementos y usa `notify_all()` para despertar a todos los consumidores a la vez.

5. Deadlock entre dos hilos

Crea 2 hilos y 2 locks (A y B). El hilo 1 adquiere A luego B. El hilo 2 adquiere B luego A. Añade `time.sleep(0.1)` entre adquisiciones para provocar el deadlock.

6. Semáforo con recursos limitados

Simula 3 impresoras compartidas. Crea 6 hilos que necesiten una impresora para imprimir (simulado con `sleep`). Usa `Semaphore(3)` para que solo 3 impriman a la vez.

★ AVANZADO 3 — Sincronización entre Hilos

★ AVANZADO 03 — Sincronización entre Hilos

1. 🎯 Banco con cuentas compartidas

Dos hilos: uno ingresa dinero, otro retira. Usa Lock para evitar saldo negativo.

Pista: Usa un Lock global. Cada hilo lo adquiere con `with lock`: antes de modificar el saldo. El hilo retirador debe comprobar `if saldo >= cantidad` antes de restar.

2. 🔍 Semáforo con timeout

Crea un semáforo donde los hilos esperen máximo 2s. Si no entran, se van.

Pista: `semaforo.acquire(blocking=True, timeout=2)` devuelve `True` si consiguió el recurso, o `False` si pasaron los 2 segundos. Úsalo para decidir si el hilo entra o se rinde.

3. 🧩 Barrera con fases múltiples

Crea 3 hilos que trabajen en 3 fases. La fase siguiente no empieza hasta que todos terminan la anterior.

Pista: Coloca `barrera.wait()` justo después del trabajo de cada fase. Así ningún hilo avanza a la siguiente fase hasta que todos han llegado al `wait()`.

4. 🐼 Condition con timeout

Un productor que produce cada 3s, consumidor que espera máximo 2s. Si no hay producto, se va.

Pista: `cond.wait(timeout=2)` devuelve `False` si el tiempo expiró sin ser notificado. Úsalo para que el consumidor salga si el productor tarda demasiado.

5. 🕒 Deadlock provocado y solución

Crea un deadlock con 2 hilos y 2 locks. Luego arréglalo adquiriendo los locks en el mismo orden.

Pista: El deadlock ocurre cuando el hilo A toma lock1 y espera lock2, mientras B toma lock2 y espera lock1. La solución: ambos hilos adquieren los locks en el mismo orden (lock1 → lock2).

6. Productor-Consumidor múltiple

2 productores y 3 consumidores compitiendo por un buffer de tamaño 5.

Pista: Usa `queue.Queue(maxsize=5)` como buffer compartido: es thread-safe y no necesita Lock. Los productores llaman a `put()` y los consumidores a `get()`.



INICIAL RESUELTO 4 — Sockets TCP

1. Servidor mínimo

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as srv:
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        print(f"Recibido: {datos.decode()}")
```

`accept()` espera un cliente (bloqueante).

2. Cliente mínimo

```
import socket
with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Hola servidor")
```

`connect()` lanza `ConnectionRefusedError` si no hay servidor.

3. Servidor eco

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(datos) # Eco
```

Devuelve exactamente lo mismo que recibe.



INICIAL POR RESOLVER 4 — Sockets

TCP

1. Servidor saludo

Crea un servidor TCP que, cuando un cliente se conecte, le envíe “Bienvenido al servidor” y luego cierre la conexión.

2. Cliente que envía y recibe

Crea un cliente que se conecte a 127.0.0.1:9000, envíe “Hola” y luego espere y muestre la respuesta del servidor.

3. Servidor contador

Crea un servidor que acepte una conexión, reciba un número como texto, lo convierta a entero, lo incremente en 1 y devuelva el resultado.



INTERMEDIO RESUELTO 4 — Sockets

TCP

4. Cliente eco

```
import socket
with socket.socket() as cli:
    cli.connect(("127.0.0.1", 9000))
    cli.sendall(b"Prueba")
    print(cli.recv(1024).decode())
```

5. SO_REUSEADDR

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    print("Servidor listo, mata y reinicia sin error")
```

Sin esta opción, al reiniciar puede dar “Address already in use”.

6. Servidor hora

```
import socket, time
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 9000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        hora = time.strftime("%H:%M:%S")
        conn.sendall(hora.encode())
```



INTERMEDIO POR RESOLVER 4 —

Sockets TCP

4. Servidor que cuenta caracteres

Crea un servidor que reciba un texto y devuelva el número de caracteres (ej: “hola” → “4”).

5. Cliente interactivo

Crea un cliente que pida texto por teclado con `input()`, lo envíe al servidor y muestre la respuesta. El bucle termina cuando el usuario escribe “salir”.

6. Servidor que registra IP

Crea un servidor que, al recibir una conexión, muestre la dirección IP y el puerto del cliente usando `conn.getpeername()`, y luego devuelva esos datos al cliente.

★ AVANZADO 4 — Sockets TCP

★ AVANZADO 04 — Sockets TCP

1. 🎯 Servidor de mayúsculas

El cliente envía texto, el servidor lo devuelve en MAYÚSCULAS.

Pista: Recibe los datos con `recv(1024).decode()`, aplica `.upper()` y envía el resultado con `sendall(...)`.

2. 🔍 Cliente con reconexión

Cliente que intenta conectar, y si falla, reintenta hasta 3 veces con 2s de espera.

Pista: Envuelve `socket.connect()` en un bucle `for` con `try/except`. Captura `ConnectionRefusedError` y `socket.timeout`, espera 2s con `time.sleep(2)` y reintenta.

3. 🧩 Servidor que maneja múltiples conexiones (sin hilos)

Usa `select.select()` para atender hasta 3 clientes en un solo hilo.

Pista: Configura el socket servidor como no bloqueante con `setblocking(False)`. `select.select()` te devuelve los sockets que tienen datos listos para leer. Si el socket listo es el servidor, acepta una nueva conexión; si es un cliente, recibe datos.

4. 🤖 Calculadora remota

El cliente envía "5+3", el servidor calcula y responde "Resultado: 8".

Pista: Usa `eval(expr)` para evaluar la expresión recibida. Envuelve en `try/except` para capturar errores (por ejemplo, división entre cero o sintaxis inválida).

5. 🕒 Timeout personalizado

Crea un servidor que cierre la conexión si el cliente no envía datos en 10 segundos.

Pista: Después de `accept()`, llama a `conn.settimeout(10)`. Captura `socket.timeout` y envía un mensaje de despedida antes de cerrar.

6. 🏰 Servidor de chat simple

Un servidor que recibe mensajes de un cliente y los reenvía a todos los demás.

Pista: Mantén una lista global de conexiones. Usa un Lock al modificar la lista. Cuando un socket recibe datos, recorre la lista y reenvía con `sendall()` a todos menos al emisor. Usa un hilo por cliente con `threading.Thread`.



INICIAL RESUELTO 5 — Sockets UDP y Protocolos

1. Servidor UDP mínimo

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    datos, direccion = srv.recvfrom(1024)
    print(f"De {direccion}: {datos.decode()}")
```

UDP no tiene `accept()`. Solo `recvfrom()`.

2. Cliente UDP mínimo

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as cli:
    cli.sendto(b"Hola UDP", ("127.0.0.1", 9001))
```

UDP no tiene `connect()`. Directamente `sendto()`.

3. Servidor UDP eco

```
import socket
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as srv:
    srv.bind(("127.0.0.1", 9001))
    datos, direccion = srv.recvfrom(1024)
    srv.sendto(datos, direccion)
```




INICIAL POR RESOLVER 5 — Sockets

UDP y Protocolos

1. Cliente UDP con entrada de usuario

Crea un cliente UDP que pida un mensaje al usuario por teclado con `input()`, lo envíe a `127.0.0.1:9001` y espere una respuesta.

2. Servidor UDP con eco personalizado

Crea un servidor UDP que escuche en `127.0.0.1:9001`. Al recibir un mensaje, responda con "Recibido: " seguido del mensaje original.

3. Servidor UDP contador

Crea un servidor UDP que lleve la cuenta de cuántos mensajes ha recibido. Al responder, incluya el número de mensaje (ej: "Mensaje #1", "Mensaje #2").



INTERMEDIO RESUELTO 5 — Sockets

UDP y Protocolos

4. Cliente HTTP manual

```
import socket
with socket.socket() as s:
    s.connect(("httpbin.org", 80))
    s.sendall(b"GET /ip HTTP/1.1\r\nHost: httpbin.org\r\nConnection: close\r\n\r\n")
    resp = b""
    while True:
        d = s.recv(4096)
        if not d: break
        resp += d
    print(resp.decode()[:500])
```

HTTP es texto sobre TCP. Mandas una petición y recibes una respuesta.

5. Compara TCP y UDP

```
import socket, time
def test_tcp():
    t = time.time()
    for _ in range(10):
        with socket.socket() as s:
            s.connect(("127.0.0.1", 9000))
            s.send(b"x")
            s.recv(1024)
    return time.time() - t
def test_udp():
    t = time.time()
    for _ in range(10):
        with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
            s.sendto(b"x", ("127.0.0.1", 9001))
            s.recvfrom(1024)
    return time.time() - t
print(f"TCP: {test_tcp():.3f}s")
print(f"UDP: {test_udp():.3f}s")
```

UDP suele ser más rápido porque no tiene handshake.

6. Mini servidor web

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 8080))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        conn.recv(1024) # Leer petición (la ignoramos)
        respuesta = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<h1>Hola mundo</h1>"
        conn.sendall(respuesta.encode())
```

Abre <http://127.0.0.1:8080> en tu navegador y verás “Hola mundo”.



INTERMEDIO POR RESOLVER 5 —

Sockets UDP y Protocolos

4. Cliente NTP manual

Crea un cliente UDP que obtenga la hora actual desde `pool.ntp.org` usando el puerto 123. Envía un paquete de 48 bytes (el primero con valor `\x1b` y el resto `\0`). Extrae el timestamp de los bytes 40 a 43 con `struct.unpack('!I', ...)` y ajústalo restando 2208988800 para convertirlo a hora Unix.

5. Servidor UDP multimensaje

Crea un servidor UDP que reciba y responda a 3 mensajes consecutivos en un bucle antes de cerrarse. Cada respuesta debe incluir el número de orden: "OK #1", "OK #2", "OK #3".

6. Cliente HTTP con parseo de cabeceras

Conéctate con un socket TCP a `example.com:80` y haz un GET a `/`. Una vez recibida la respuesta completa, parsea las cabeceras y muestra los valores de `Content-Type` y `Content-Length`.

★ AVANZADO 5 — Sockets UDP y Protocolos

★ AVANZADO 05 — Sockets UDP y Protocolos

1. 🎯 Ping UDP

Cliente manda "PING", servidor responde "PONG". Mide cuánto tarda.

Pista: Necesitas dos funciones (servidor y cliente) ejecutándose en paralelo. Usa `threading.Thread` con `daemon=True` para lanzar el servidor. Mide el tiempo con `time.time()` antes y después del intercambio de mensajes.

2. 🔍 Broadcast UDP

El servidor escucha en todas las interfaces y responde a cualquiera.

Pista: El servidor debe escuchar en "0.0.0.0" para aceptar conexiones de cualquier interfaz. Usa un bucle infinito con `recvfrom()` y responde con `sendto()` a la dirección de cada cliente.

3. 🧩 HTTP desde cero con parseo

Cliente HTTP manual que parsea el código de estado y las cabeceras.

Pista: Después de recibir la respuesta HTTP completa, separa las cabeceras del cuerpo con `partition("\r\n\r\n")`. La primera línea de las cabeceras contiene el código de estado (ej: HTTP/1.1 200 OK).

4. 🤖 Mini navegador web

Crea una función que descargue el HTML de una URL y lo guarde en un archivo.

Pista: Extrae el host y la ruta de la URL. Conéctate al puerto 80 del host, envía un GET con `Connection: close`. Tras recibir toda la respuesta, separa el cuerpo de las cabeceras con `partition(b"\r\n\r\n")` y escribe el cuerpo a un archivo.

5. 🕒 Comparativa TCP vs UDP

Mide el tiempo de 100 mensajes con TCP y con UDP en local.

Pista: Necesitas servidores TCP y UDP separados ejecutándose en hilos. El servidor TCP requiere `accept()` por cada mensaje; el UDP solo `recvfrom()`. Mide el tiempo total para 100 intercambios en cada protocolo y compara.

6. 🏠 Servidor HTTP simple

Crea un servidor TCP que entienda peticiones HTTP GET y sirva respuestas.

Pista: Con `accept()` obtienes la conexión. Lee la petición con `recv()` y parsea la primera línea (GET /ruta HTTP/1.1). Según la ruta, devuelve distinto contenido HTML con cabeceras HTTP válidas incluyendo Content-Length.



INICIAL RESUELTO 6 — APIs REST y HTTP

1. GET a GitHub

```
import requests
resp = requests.get("https://api.github.com/users/python")
print(resp.status_code) # 200
```

2. Mostrar JSON

```
import requests
resp = requests.get("https://api.github.com/users/python")
print(resp.json())
```

`resp.json()` convierte el JSON a diccionario Python.

3. Nombre real

```
import requests
datos = requests.get("https://api.github.com/users/python").json()
print(datos["name"]) # "Python"
```

INICIAL POR RESOLVER 6 — APIs REST y HTTP

1. GET a JSONPlaceholder

Usa `requests.get` para obtener el post con ID 1 de `https://jsonplaceholder.typicode.com/posts/1`. Muestra el `status_code`.

2. Título del post

Del ejercicio anterior, muestra el campo "title" del post obtenido usando `resp.json()`.

3. Contar usuarios

Haz GET a `https://jsonplaceholder.typicode.com/users` y muestra cuántos usuarios hay (la longitud de la lista).



INTERMEDIO RESUELTO 6 — APIs REST y HTTP

4. Código 404

```
import requests
resp = requests.get("https://api.github.com/users/usuarioquenoexiste123")
print(resp.status_code) # 404
```

El servidor devuelve 404. No lanza excepción a menos que uses `raise_for_status()`.

5. GET con parámetros

```
import requests
resp = requests.get("https://jsonplaceholder.typicode.com/posts", params={"userId": 1})
posts = resp.json()
print(f"El usuario 1 tiene {len(posts)} posts")
```

`params` construye la query string automáticamente.

6. POST simple

```
import requests
resp = requests.post("https://jsonplaceholder.typicode.com/posts",
                    json={"title": "test", "body": "test", "userId": 1})
print(resp.status_code) # 201 Created
```

201 = "Created". El recurso se ha creado correctamente.



INTERMEDIO POR RESOLVER 6 — APIs REST y HTTP

4. PUT para actualizar

Haz PUT a `https://jsonplaceholder.typicode.com/posts/1` con `json={"title": "nuevo titulo", "body": "nuevo cuerpo", "userId": 1}`. Muestra el código de estado y el título actualizado.

5. DELETE un recurso

Haz DELETE a `https://jsonplaceholder.typicode.com/posts/1`. ¿Qué código de estado devuelve un borrado exitoso?

6. HEAD request

Usa `requests.head()` para pedir solo las cabeceras de `https://api.github.com`. Muestra los valores de `Content-Type` y `X-RateLimit-Remaining` de las cabeceras.

★ AVANZADO 6 — APIs REST y HTTP

★ AVANZADO 06 — APIs REST y HTTP

1. 🎯 Info de usuario de GitHub

Obtén los datos del usuario “python” y muestra: nombre, bio, repos públicos, seguidores, fecha de creación.

Pista: La API de GitHub (<https://api.github.com/users/python>) devuelve un JSON con campos como `name`, `bio`, `public_repos`, `followers` y `created_at`. Usa `resp.json()` para acceder a cada campo.

2. 🔍 Buscador de repositorios

Busca repositorios de Python con más de 1000 estrellas. Muestra nombre y estrellas.

Pista: La API de búsqueda de GitHub (`/search/repositories`) acepta `q` como parámetro. Usa `"python stars:>1000"` para filtrar. Ordena por stars con `sort=stars` y limita a 5 resultados con `per_page=5`.

3. 🔄 API con paginación

La API de GitHub página los resultados. Obtén los primeros 20 repos del usuario “python” (2 páginas de 10).

Pista: Usa los parámetros `page` y `per_page` en cada petición a <https://api.github.com/users/{usuario}/repos>. Itera sobre las páginas que necesites y acumula los resultados en una lista.

4. 🎭 POST y respuesta

Crea un post en JSONPlaceholder, luego haz GET para verificarlo.

Pista: JSONPlaceholder es una API de pruebas. El POST devuelve el recurso creado con un ID (101). Luego puedes hacer GET a <https://jsonplaceholder.typicode.com/posts/101> para verificar la creación.

5. 🕒 Tiempo de respuesta

Mide el tiempo de respuesta de 5 APIs distintas. ¿Cuál es más rápida?

Pista: Mide el tiempo con `time.time()` antes y después de cada `requests.get()`. Prueba varias APIs gratuitas (GitHub, JSONPlaceholder, httpbin, ReqRes) y compara los tiempos en milisegundos.

6. 🏠 Cliente de API con caché

Crea un cliente que cachee respuestas en un diccionario para no repetir peticiones.

Pista: Usa un diccionario global donde la clave sea la URL. Guarda el dato y el timestamp. Antes de hacer una petición, comprueba si la URL está en caché y si el tiempo transcurrido desde que se guardó es menor al TTL.



INICIAL RESUELTO 7 — APIs Comerciales

1. Clima de tu ciudad

```
import requests
API_KEY = "tu_api_key_real"
params = {"q": "Sevilla", "appid": API_KEY, "units": "metric"}
resp = requests.get("https://api.openweathermap.org/data/2.5/weather", params=params)
print(resp.json()["main"]["temp"], "°C")
```

Necesitas una API key real de OpenWeatherMap (gratuita).

2. Sensación térmica

```
import requests
API_KEY = "tu_api_key"
params = {"q": "Madrid", "appid": API_KEY, "units": "metric"}
datos = requests.get("https://api.openweathermap.org/data/2.5/weather",
params=params).json()
print(f"Sensación: {datos['main']['feels_like']}°C")
```

3. Error 401

```
import requests
resp = requests.get("https://api.openweathermap.org/data/2.5/weather",
                    params={"q": "Sevilla", "appid": "falsa", "units": "metric"})
print(resp.status_code) # 401
```

API key incorrecta → 401 Unauthorized.



INICIAL POR RESOLVER 7 — APIs Comerciales

1. Humedad actual

Usa OpenWeatherMap para obtener el clima de tu ciudad. Muestra la humedad (`humidity`) del campo `main`.

2. Descripción del clima

Del mismo JSON, muestra la descripción del tiempo (campo `description` dentro de `weather`).

3. Temperatura mínima y máxima

Muestra la temperatura mínima (`temp_min`) y máxima (`temp_max`) del pronóstico actual de tu ciudad.



INTERMEDIO RESUELTO 7 — APIs Comerciales

4. .env básico

```
from dotenv import load_dotenv
import os
load_dotenv()
print(os.getenv("OPENWEATHER_API_KEY")) # "test123"
```

Crea un .env con: OPENWEATHER_API_KEY=test123

5. Chat con GPT

```
from openai import OpenAI
from dotenv import load_dotenv
import os
load_dotenv()
cliente = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
resp = cliente.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": "¿Qué es Python?"}]
)
print(resp.choices[0].message.content)
```

6. Timeout

```
import requests
try:
    resp = requests.get("https://192.0.2.1", timeout=3)
except requests.exceptions.Timeout:
    print("Timeout – el servidor no responde")
```

192.0.2.1 es una IP de test que nunca responde.



INTERMEDIO POR RESOLVER 7 — APIs Comerciales

4. Clima con .env para dos ciudades

Carga tu API key de OpenWeatherMap desde un archivo .env con python-dotenv. Obtén el clima de dos ciudades distintas y compara sus temperaturas.

5. GPT con temperatura creativa

Usa la API de OpenAI para pedir una historia corta sobre Python. Ajusta el parámetro `temperature=0.9` para obtener una respuesta más creativa.

6. Capturar errores HTTP

Haz una petición a `https://httpbin.org/status/404` y otra a `https://httpbin.org/status/500`. Captura los errores usando `raise_for_status()` dentro de un bloque `try/except`.

★ AVANZADO 7 — APIs Comerciales

★ AVANZADO 07 — APIs Comerciales

1. 🎯 Pronóstico extendido

Usa OpenWeatherMap para obtener el pronóstico de 5 días de tu ciudad.

Pista: OpenWeatherMap tiene un endpoint `/forecast` que devuelve datos cada 3 horas. Usa `lang=es` para descripciones en español. Para obtener un dato por día, salta 8 elementos ($24 \text{ horas} \div 3 \text{ horas} = 8 \text{ intervalos}$).

2. 🔍 Múltiples ciudades

Obtén el clima de 5 ciudades a la vez usando un solo bucle.

Pista: Itera sobre una lista de ciudades y haz una petición a OpenWeatherMap por cada una. Añade `time.sleep(0.2)` entre peticiones para evitar el rate limit. Cada respuesta contiene `main.temp` y `weather[0].description`.

3. 🧩 GPT: explicador automático

Pregunta a GPT-3.5 que explique 3 conceptos de Python: Lock, Semaphore, Barrier.

Pista: La API de chat de OpenAI recibe una lista de mensajes. Puedes iterar sobre una lista de conceptos y pedir una explicación corta para cada uno, limitando la respuesta con `max_tokens`.

4. 🗣️ Conversación con contexto

Chat con GPT que recuerda el historial de la conversación.

Pista: Mantén una lista mensajes que acumule cada interacción. Empieza con un mensaje `{"role": "system"}` para definir el rol. En cada turno, añade el mensaje del usuario y la respuesta del asistente para mantener el contexto.

5. 🕒 Monitor de uptime

Comprueba cada 30s si una API responde. Si falla 3 veces seguidas, alerta.

Pista: Un bucle infinito con `time.sleep(30)` dentro. Un contador de fallos consecutivos se incrementa en cada error y se reinicia al obtener una respuesta OK. Cuando supere el máximo, muestra una alerta.

6. 🏗️ Agregador de APIs

Llama a 3 APIs distintas y combina sus respuestas en un solo resumen.

Pista: Define una función que llame a varias APIs y guarde los datos relevantes en un diccionario. Cada API aporta una clave diferente. Usa GitHub para datos de usuario, JSONPlaceholder para el conteo de posts y httpbin para la IP.



INICIAL RESUELTO 8 — Hash y Cifrado Clásico

1. MD5 de “hola”

```
import hashlib
print(hashlib.md5(b"hola").hexdigest())
# a47c1382b0e5e7b7f6e6349e4ff93e05
```

2. SHA256 de “python”

```
import hashlib
h = hashlib.sha256(b"python").hexdigest()
print(h)
print(f"Longitud: {len(h)} caracteres") # 64
```

SHA256 siempre devuelve 64 caracteres hexadecimales (256 bits).

3. Compara hashes

```
import hashlib
texto = b"Hola mundo"
print(f"MD5: {len(hashlib.md5(texto).hexdigest())} chars (128 bits)")
print(f"SHA1: {len(hashlib.sha1(texto).hexdigest())} chars (160 bits)")
print(f"SHA256: {len(hashlib.sha256(texto).hexdigest())} chars (256 bits)")
```

MD5 → 32, SHA1 → 40, SHA256 → 64 caracteres.



INICIAL POR RESOLVER 8 — Hash y Cifrado Clásico

1. SHA1 de tu nombre

Calcula el hash SHA1 de tu nombre (en minúsculas) usando `hashlib.sha1()`. Muestra el resultado en hexadecimal con `.hexdigest()`.

2. MD5 de una frase

Calcula el hash MD5 de la frase "Python es genial". Muestra el resultado en hexadecimal.

3. Compara “Hola” vs “hola”

Calcula SHA256 de "Ho1a" y de "ho1a". ¿Los hashes son iguales o diferentes? ¿Por qué?



INTERMEDIO RESUELTO 8 — Hash y Cifrado Clásico

4. Hash de un número

```
import hashlib
h1 = hashlib.sha256(b"12345").hexdigest()
h2 = hashlib.sha256(b"12346").hexdigest()
print(f"12345: {h1}")
print(f"12346: {h2}")
print(f"¿Son parecidos? No, efecto avalancha.")
```

Un mínimo cambio produce un hash completamente diferente.

5. César básico

```
def cifrar(texto, desp):
    res = ""
    for c in texto:
        if c.isalpha():
            base = ord('A') if c.isupper() else ord('a')
            res += chr((ord(c) - base + desp) % 26 + base)
        else:
            res += c
    return res
print(cifrar("Hola", 3)) # "Krod"
```

6. Descifrar César

```
def descifrar(texto, desp):
    return cifrar(texto, -desp)
print(descifrar("Krod", 3)) # "Hola"
```

Descifrar es cifrar con desplazamiento negativo.



INTERMEDIO POR RESOLVER 8 — Hash y Cifrado Clásico

4. Hash de un archivo de texto

Escribe "Hola mundo" en un archivo `mensaje.txt`. Lee su contenido en modo binario y calcula el hash SHA256. Verifica que coincida con hacer hash directamente del mismo string.

5. Cifrado César con espacios

Implementa una función `cifrar_cesar(texto, desplazamiento)` que cifre frases completas respetando espacios y signos de puntuación sin modificarlos. Pruébala con "Hola mundo" con desplazamiento 5.

6. Descifrado con mayúsculas y minúsculas

Implementa descifrado César que maneje tanto mayúsculas como minúsculas. Descifra "Nkrru" sabiendo que fue cifrado con desplazamiento 2.

★ AVANZADO 8 — Hash y Cifrado Clásico

★ AVANZADO 08 — Hash y Cifrado Clásico

1. 🎯 Verificador de integridad

Calcula el SHA256 de un archivo y compáralo con un hash esperado.

Pista: Lee el archivo en modo binario ("rb") y pásalo directamente a `hashlib.sha256().hexdigest()`. Compara el hash calculado con el hash esperado del mismo contenido. Si coinciden, el archivo no ha sido modificado.

2. 🔍 Fuerza bruta César

Dado un texto cifrado con César (desplazamiento desconocido), prueba los 25 desplazamientos y muestra solo los que tengan palabras en español.

Pista: Para cada desplazamiento del 1 al 25, aplica el descifrado César. Usa un conjunto de palabras comunes en español (como "hola", "mundo", "python") para filtrar resultados. Si el texto descifrado contiene alguna palabra conocida, es probable que sea el correcto.

3. 🧩 Hash con salt

Añade un salt aleatorio al hash de una contraseña para evitar tablas rainbow.

Pista: Genera un salt de 16 bytes con `os.urandom(16)`. El hash final es `salt + sha256(salt + password.encode()).digest()`. Para verificar, extrae los primeros 16 bytes (el salt) y repite el cálculo. Si dos usuarios tienen la misma contraseña, sus hashes serán distintos gracias al salt.

4. 🤖 Comparación gráfica de avalancha

Muestra cómo un bit de diferencia en la entrada cambia completamente el hash.

Pista: Convierte ambos hashes a hexadecimal con `hexdigest()` y compáralos carácter por carácter. Cuenta cuántos caracteres son diferentes. El efecto avalancha debería mostrar aproximadamente el 50% de caracteres distintos aunque la entrada cambie solo un bit.

5. 🕒 Velocidad de hashes

Compara cuánto tarda MD5 vs SHA1 vs SHA256 en hashear 1 millón de veces.

Pista: Usa un bucle de 1 millón de iteraciones llamando a cada función de hash (`hashlib.md5`, `hashlib.sha1`, `hashlib.sha256`). Mide el tiempo total con `time.time()` antes y después. MD5 es el más rápido; SHA256 el más lento pero más seguro.

6. 🏠 Mini gestor de contraseñas

Guarda contraseñas con hash + salt en un archivo JSON. Permite registro y login.

Pista: Almacena cada usuario en un JSON con el formato `salt.hex() + hashlib.sha256(salt + password.encode()).hexdigest()`. Para verificar, extrae el salt (primeros 32 caracteres hexadecimales), conviértelo a bytes con `bytes.fromhex()` y recalcula el hash con la contraseña proporcionada.



INICIAL RESUELTO 9 — Cifrado Moderno

1. AES: cifrar mensaje

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
clave = get_random_bytes(32)
cifrador = AES.new(clave, AES.MODE_EAX)
texto_cifrado, tag = cifrador.encrypt_and_digest(b"Hola AES")
print(f"Cifrado: {texto_cifrado.hex()}")
```

pycryptodome debe estar instalado: `pip install pycryptodome`.

2. AES: descifrar

```
cifrador2 = AES.new(clave, AES.MODE_EAX, nonce=cifrador.nonce)
original = cifrador2.decrypt(texto_cifrado)
print(f"Original: {original.decode()}")
```

Necesitas la misma clave y el mismo nonce para descifrar.

3. RSA: generar claves

```
from Crypto.PublicKey import RSA
clave = RSA.generate(2048)
print(clave.publickey().export_key().decode()[:50] + "...")
```

La clave pública se puede compartir. La privada NO.



INICIAL POR RESOLVER 9 — Cifrado Moderno

1. AES modo ECB

Cifra el mensaje `b"0123456789ABCDEF"` (16 bytes exactos) usando AES en modo ECB. Muestra el resultado en hexadecimal.

2. Nonce y tag

Cifra `b"Hola mundo con AES"` usando AES modo EAX. Imprime por pantalla las longitudes del nonce y del tag (en bytes).

3. RSA: exportar clave

Genera un par de claves RSA de 2048 bits. Exporta la clave pública a formato PEM (string) y muestra sus primeros 40 caracteres y sus últimos 40 caracteres.



INTERMEDIO RESUELTO 9 — Cifrado Moderno

4. RSA: cifrar y descifrar

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
clave = RSA.generate(2048)
cifrador = PKCS1_OAEP.new(clave.publickey())
c = cifrador.encrypt(b"Hola RSA")
descifrador = PKCS1_OAEP.new(clave)
print(descifrador.decrypt(c).decode())
```

Solo la clave privada puede descifrar lo que cifró la pública.

5. Firma digital

```
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.PublicKey import RSA
clave = RSA.generate(2048)
mensaje = b"Este mensaje es de Ana"
h = SHA256.new(mensaje)
firma = pkcs1_15.new(clave).sign(h)
try:
    pkcs1_15.new(clave.publickey()).verify(h, firma)
    print("✅ Firma válida")
except:
    print("❌ Firma inválida")
```

6. RBAC simple

```
permisos = {"admin": ["leer", "escribir"], "user": ["leer"]}
def puede(usuario, accion):
    return accion in permisos.get(usuario["rol"], [])
print(puede({"rol": "admin"}, "escribir")) # True
print(puede({"rol": "user"}, "escribir")) # False
```



INTERMEDIO POR RESOLVER 9 —

Cifrado Moderno

4. Cifrado híbrido simplificado

Genera una clave AES de 32 bytes y cifra `b"El cifrado hibrido funciona"`. Luego cifra esa clave AES con una clave RSA pública. Descifra en orden inverso y verifica el mensaje original.

5. Firma alterada

Firma digitalmente el mensaje `b"Transferencia de 500€"`. Modifica UN byte de la firma y comprueba que la verificación falla con `pkcs1_15.new(...).verify(...)`.

6. RBAC con permisos cifrado

Crea un sistema RBAC con 3 roles: `admin` (cifrar, descifrar, firmar), `usuario` (cifrar, firmar), `invitado` (solo cifrar). Implementa la función `puede(usuario, accion)` y pruébala con cada rol.

★ AVANZADO 9 — Cifrado Moderno

★ AVANZADO 09 — Cifrado Moderno

1. 🎯 Cifrar archivo completo

Cifra un archivo de texto con AES y guarda el resultado. Luego descifralo.

Pista: Abre el archivo en modo binario `"rb"` / `"wb"`. Usa `AES.MODE_EAX`, cifra con `encrypt_and_digest` y guarda `nonce + tag + cifrado`. Para descifrar, separa los tres componentes y usa `decrypt_and_verify`.

2. 🔍 RSA: cifrar mensajes largos

RSA solo cifra ~190 bytes. Intenta cifrar un mensaje de 300 bytes. ¿Qué pasa? ¿Cómo lo arreglas?

Pista: Captura el `ValueError` para ver el mensaje de error. Piensa en combinar RSA con un cifrado simétrico como AES (cifrado híbrido).

3. 🧩 Intercambio de claves simulado

Simula el intercambio de claves entre Ana y Bob: Ana genera RSA, Bob cifra una clave AES con RSA pública de Ana.

Pista: Define una clave AES de 32 bytes del lado de Bob. Bob cifra el mensaje con AES, luego cifra la clave AES con la RSA pública de Ana. Envía los 4 componentes (`clave_AES_cifrada`, `nonce`, `tag`, `cifrado`). Ana descifra en orden inverso.

4. 🎭 Firma con verificación de integridad

Firma un mensaje, modifica el mensaje, y muestra que la verificación falla.

Pista: Usa `SHA256.new(mensaje)` y `pkcs1_15.new(clave).sign(h)`. Después de firmar el original, crea un segundo mensaje modificado y verifícalo con la misma firma. La verificación debe lanzar `ValueError` o `TypeError`.

5. 🕒 RSA vs AES benchmark

Mide cuánto tarda cifrar el mismo mensaje con RSA y AES. La diferencia es abismal.

Pista: Usa `time.time()` antes y después de un bucle de 100 cifrados RSA y otro de 1000 cifrados AES. Con `time.time() - t` obtienes los segundos. Multiplica por 1000 para milisegundos.

6. 🏰 Sistema de cifrado de extremo a extremo

Simula un chat cifrado: cada usuario tiene su par RSA, y los mensajes se cifran con AES + RSA híbrido.

Pista: Crea una clase `Usuario` con `nombre` y `clave_rsa`. Un método `cifrar_para(mensaje, destinatario)` que genera clave AES, cifra el mensaje y la clave. Otro método `descifrar(nonce, tag, clave_aes_cifrada, cifrado)` que hace el proceso inverso.



INICIAL RESUELTO 10 — Servidores Concurrentes

1. Servidor secuencial

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    conn, addr = srv.accept()
    with conn:
        datos = conn.recv(1024)
        conn.sendall(b"OK")
```

Solo atiende UN cliente y termina.

2. Bucle de clientes

```
import socket
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        with conn:
            conn.recv(1024)
            conn.sendall(b"OK")
```

Clientes uno tras otro. Si uno tarda, los demás esperan.

3. Hilo por cliente

```
import socket, threading
def atender(conn, addr):
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
with socket.socket() as srv:
    srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        threading.Thread(target=atender, args=(conn, addr)).start()
```

Cada cliente en su propio hilo. Todos se atienden en paralelo.



INICIAL POR RESOLVER 10 — Servidores Concurrentes

1. Servidor eco básico

Crea un servidor TCP que reciba datos de un cliente y los devuelva exactamente igual (eco). Sin bucle, atiende un solo cliente y termina.

2. Servidor eco con bucle

Modifica el servidor anterior para que acepte clientes en un bucle infinito, dando eco a cada uno.

3. Cliente eco

Crea un cliente TCP que se conecte al servidor, envíe b"Hola eco" y muestre la respuesta recibida.



INTERMEDIO RESUELTO 10 —

Servidores Concurrentes

4. ThreadPool

```
import socket, concurrent.futures

def atender(conn, addr):
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")

with socket.socket() as srv, concurrent.futures.ThreadPoolExecutor(3) as pool:
    srv.bind(("127.0.0.1", 5000))
    srv.listen()
    while True:
        conn, addr = srv.accept()
        pool.submit(atender, conn, addr)
```

Máximo 3 hilos. Los clientes adicionales esperan en cola.

5. Lanzador de clientes

```
import socket, threading

def cliente(id):
    with socket.socket() as s:
        s.connect(("127.0.0.1", 5000))
        s.sendall(f"Soy {id}".encode())
        print(s.recv(1024).decode())

hilos = [threading.Thread(target=cliente, args=(i,)) for i in range(5)]
for h in hilos: h.start()
for h in hilos: h.join()
```

Lanza 5 clientes a la vez.

6. Contador de conexiones

```
contador = 0
lock = threading.Lock()

def atender(conn, addr):
    global contador
    with lock:
        contador += 1
        print(f"Total: {contador}")
    with conn:
        conn.recv(1024)
        conn.sendall(b"OK")
```

El Lock protege la variable compartida contador.



INTERMEDIO POR RESOLVER 10 —

Servidores Concurrentes

4. Servidor con timeout

Implementa un servidor que cierre la conexión si el cliente no envía datos en 5 segundos (usa `socket.settimeout` o `select`).

5. Cliente con timeout

Crea un cliente que intente conectarse a `127.0.0.1:9999` con timeout de 2 segundos. Captura la excepción `socket.timeout` y muestra “Servidor no disponible”.

6. Contador de bytes totales

Añade un contador global de bytes recibidos (protegido con Lock) al servidor multihilo. Cada vez que un cliente envía datos, suma los bytes y muestra el total acumulado.

★ AVANZADO 10 — Servidores Concurrentes

★ AVANZADO 10 — Servidores Concurrentes

1. 🎯 Servidor con límite de clientes

Crea un servidor que acepte máximo 3 clientes. El cuarto recibe “Servidor completo” y se cierra.

Pista: Usa una variable global `clientes_activos` protegida con `Lock`. Al aceptar un cliente, comprueba si ya se alcanzó el máximo; si es así, envía el mensaje y cierra sin incrementar el contador. Decrementa al terminar.

2. 🔍 Prueba de carga

Lanza 20 clientes simultáneos contra tu servidor y mide cuánto tardan todos en recibir respuesta.

Pista: Usa un diccionario `resultados` compartido para guardar el tiempo de cada cliente. Cada cliente mide `time.time()` antes y después de la conexión. Lanza 20 hilos, haz `join` a todos y calcula estadísticas (total, media, exitosos).

3. 🧩 Servidor con cola de espera

Cuando el pool está lleno, los clientes entran en una cola. Cuando un hilo se libera, atiende al siguiente.

Pista: Usa `queue.Queue` para encolar las conexiones aceptadas. Crea `N` hilos trabajadores (daemon) que saquen elementos de la cola con `cola.get()` y atiendan al cliente. El hilo principal solo acepta y encola.

4. 🤖 Estado del servidor

Añade un endpoint especial: si el cliente envía “STATUS”, el servidor responde con número de conexiones activas.

Pista: Mantén un contador activas con Lock. En la función de atención, parsea el comando: si es "STATUS", responde con el valor actual del contador; si no, responde "OK". Incrementa al entrar, decrementa al salir.

5. 🕒 Heartbeat en servidor

El servidor tiene un hilo heartbeat que imprime “❤️ Servidor vivo — N conexiones” cada 5s.

Pista: Crea un hilo `daemon=True` con un bucle infinito que haga `time.sleep(5)` y luego imprima el estado usando el Lock para leer el contador de conexiones activas.

6. 🏗️ Balanceador de carga simple

Crea un “balanceador” que recibe peticiones y las distribuye entre 2 servidores workers.

Pista: Crea dos workers en los puertos 5001 y 5002 (cada uno en su hilo). El balanceador en el puerto 5000 acepta conexiones y las reenvía al worker actual haciendo de proxy: lee del cliente, envía al worker, recibe la respuesta y la reenvía al cliente. Alterna entre workers con un índice round-robin.



INICIAL RESUELTO 11 — Asyncio y Disponibilidad

1. Corrutina básica

```
import asyncio
async def hola():
    print("Hola")
    await asyncio.sleep(1)
    print("Mundo")
asyncio.run(hola())
```

await cede el control. asyncio.run() crea el event loop.

2. Dos corrutinas

```
import asyncio
async def tarea(n, s):
    await asyncio.sleep(s)
    print(f"Tarea {n} terminó")
async def main():
    await asyncio.gather(tarea(1, 1), tarea(2, 2))
asyncio.run(main())
```

gather ejecuta ambas “a la vez”. Terminan en ~2s total.

3. asyncio.sleep

```
import asyncio
async def contar():
    for i in range(1, 4):
        print(i)
        await asyncio.sleep(1)
asyncio.run(contar())
```



INICIAL POR RESOLVER 11 — Asyncio y Disponibilidad

1. Mensaje diferido

Crea una corrutina `async def preparar()` que imprima “Preparando...”, espere 2 segundos con `await asyncio.sleep(2)`, e imprima “¡Listo!”. Ejecútala con `asyncio.run()`.

2. Saludo y despedida

Crea dos corrutinas: `saludar()` (espera 0.5s y dice “Hola”) y `despedirse()` (espera 1s y dice “Adiós”). Ejecuta ambas con `asyncio.gather()`.

3. Temporizador

Crea una corrutina que imprima “tic” cada 2 segundos, 4 veces. Usa `asyncio.sleep(2)` dentro de un bucle `for`.



INTERMEDIO RESUELTO 11 — Asyncio y Disponibilidad

4. Heartbeat simple

```
import asyncio
async def heartbeat():
    while True:
        print("❤️")
        await asyncio.sleep(2)
async def main():
    asyncio.create_task(heartbeat())
    await asyncio.sleep(5)
    print("Main terminó")
asyncio.run(main())
```

create_task lanza en segundo plano.

5. Timeout con asyncio

```
import asyncio
async def lento():
    await asyncio.sleep(10)
    return "Listo"
async def main():
    try:
        r = await asyncio.wait_for(lento(), timeout=3)
        print(r)
    except asyncio.TimeoutError:
        print("Timeout!")
asyncio.run(main())
```

wait_for lanza TimeoutError si la corrutina excede el tiempo.

6. Backoff

```
import asyncio
async def conectar(intentos=3):
    for i in range(intentos):
        espera = 2 ** i
        try:
            print(f"Intento {i+1}, espera {espera}s")
            # await asyncio.open_connection(...)
            raise ConnectionRefusedError # Simular error
        except ConnectionRefusedError:
            await asyncio.sleep(espera)
    print("No se pudo conectar")
asyncio.run(conectar())
```

Backoff exponencial: 1s, 2s, 4s.



INTERMEDIO POR RESOLVER 11 —

Asyncio y Disponibilidad

4. Backoff exponencial

Crea una función asíncrona `conectar()` que intente conectarse 4 veces con backoff (1s, 2s, 4s, 8s). Simula el fallo lanzando `ConnectionRefusedError`. Si falla, imprime “Servicio no disponible”.

5. Timeout con respaldo

Crea una corrutina `lenta()` que tarde 8 segundos. Usa `asyncio.wait_for` con timeout de 5s. Si salta `TimeoutError`, ejecuta una corrutina de respaldo que devuelva “Resultado en caché”.

6. Dos heartbeats

Crea dos corrutinas heartbeat: `hb_a()` imprime “💖 A” cada 3s, `hb_b()` imprime “💖 B” cada 5s. La función `main` las lanza con `asyncio.create_task` y espera 12 segundos.

★ AVANZADO 11 — Asyncio y Disponibilidad

★ AVANZADO 11 — Asyncio y Disponibilidad

1. 🎯 Web scraper asíncrono

Descarga 5 URLs a la vez con `asyncio` y `httpx`. Compara el tiempo con una versión síncrona.

Pista: Usa `httpx.AsyncClient` dentro de cada corrutina. Crea una lista de tareas con `[descargar(u) for u in urls]` y ejecútalas con `asyncio.gather(*tareas)`. Mide el tiempo total con `time.time()`.

2. 🔍 Servidor asyncio con heartbeat

Servidor asyncio que imprime “❤️ Vivo — N conexiones” cada 5s.

Pista: Usa `asyncio.start_server` para el servidor TCP. Crea una corrutina `heartbeat` con un bucle infinito `while True: await asyncio.sleep(5); ...`. Lánzala con `asyncio.create_task` antes de iniciar el servidor.

3. 🧩 Semáforo asyncio

Limita a 3 descargas simultáneas usando `asyncio.Semaphore`.

Pista: Crea `sem = asyncio.Semaphore(3)`. Dentro de la función de descarga, usa `async with sem:` para que solo 3 corrutinas puedan ejecutar el bloque a la vez. Usa `httpbin.org/delay/{n}` para simular descargas lentas.

4. 🤖 Timeout con fallback

Intenta descargar de un servidor principal. Si tarda más de 2s, usa un servidor de respaldo.

Pista: Envuelve la llamada a la corrutina principal con `asyncio.wait_for(descargar_principal(), timeout=2)`. Captura `asyncio.TimeoutError` y en el `except` ejecuta la corrutina de respaldo.

5. 🕒 Monitoreo de servidores

3 servidores simulados. Un monitor asíncrono comprueba su estado cada 3s.

Pista: Usa un diccionario `estados = {"Server-A": True, ...}`. Cada servidor es una corrutina que cambia su estado aleatoriamente cada ~5-15s. El monitor lee el diccionario cada 3s y reporta servidores caídos.

6. 🗨️ Chat asíncrono

Crea un servidor de chat con `asyncio` que reenvíe mensajes a todos los clientes conectados.

Pista: Mantén un conjunto `clientes` con los objetos `writer` de cada conexión. Crea una función `broadcast(mensaje, emisor=None)` que itere sobre una copia del conjunto y envíe a todos excepto al emisor. Usa `asyncio.wait_for(reader.read(1024), timeout=60)` para detectar desconexiones.
